



**AURELIA: AN AI-POWERED GAMIFIED FRAMEWORK FOR
ENHANCING THE LEARNING EXPERIENCE**

BY

IKEDE DIVINE OGHENEKENO

(ADUN/FS/21/587)

AKPITUREN ERONMWON SOPHIA

(ADUN/FS/22/613)

SOFTWARE ENGINEERING PROGRAMME

DEPARTMENT OF COMPUTING SCIENCES

FACULTY OF SCIENCE

ADMIRALTY UNIVERSITY OF NIGERIA (ADUN), DELTA STATE

**PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF COMPUTING
SCIENCES IN PARTIAL FUFILMENT OF THE ADUN REQUIREMENT FOR THE
AWARD OF BACHELOR OF SCIENCE DEGREE (B.Sc) IN SOFTWARE
ENGINEERING.**

SEPTEMBER, 2025

CERTIFICATION

This is to certify that this project entitled “**AURELIA: AN AI-POWERED GAMIFIED FRAMEWORK FOR ENHANCING THE LEARNING EXPERIENCE**” submitted in partial fulfillment of the requirement for the award of the degree of **BSc Software Engineering** to the department of **computing sciences**, **Admiralty University Of Nigeria** is a Bonafide record of the work done by **IKEDE DIVINE OGHENEKENO and AKPITUREN ERONMWON SOPHIA** with matriculation numbers **ADUN/FS/21/587 and ADUN/FS/22/613**

.....

IKEDE DIVINE OGHENEKENO

.....

DATE

.....

AKPITUREN ERONMWON SOPHIA

.....

DATE

DEDICATION

First and foremost, we dedicate this project to God Almighty, whose wisdom, strength, and guidance have been our greatest source of inspiration throughout this journey. We also dedicate this work to our families and loved ones, whose unwavering support, patience, belief and encouragement have been invaluable. Their belief in us has kept us motivated and determined to achieve our goals. Finally, we dedicate this project to all learners and educators striving for excellence in education. May this work contribute to making learning more accessible, engaging, and effective for individuals of all backgrounds and abilities.

ACKNOWLEDGMENTS

We sincerely express our profound gratitude to Almighty God for His unfailing grace, wisdom, strength, and provision that made it possible for us to successfully complete this project. Without His guidance and blessings, this achievement would not have been possible.

Our heartfelt appreciation goes to our project supervisor, Dr. Ezekwe Chinwe, whose support and professional insights were invaluable throughout this work. Her patience, constructive feedback, constant encouragement, and dedication gave us the confidence and direction needed to carry this research to completion. We are truly grateful for her tireless efforts and commitment towards our academic progress.

We also extend our sincere appreciation to the Head of Department, Assoc. Prof. Ifeoma Ejinkonye, together with all the lecturers of the Department of Computing Sciences. Their leadership, guidance, knowledge impartation, and commitment to excellence have greatly influenced our growth, both academically and personally. The skills, discipline, and values instilled in us during our course of study provided the foundation on which this project stands.

Special thanks also go to our parents, whose unwavering support, encouragement, and sacrifices have been a constant source of strength. Their prayers, love, and belief in our abilities have kept us motivated and focused on our goals.

We equally acknowledge our friends and contributors who, in different ways, helped, encouragement, and useful suggestions that enriched this work. Their input, whether big or small, added value to the success of this project.

Finally, we recognize the role of modern digital tools, particularly ChatGPT and DiagramGPT, which provided guidance, clarity, and helpful resources in the process of developing this research.

ABSTRACT

The growth of digital education has expanded access to learning but also introduced challenges such as information overload, low motivation, and limited support for diverse learning styles. Learners often struggle to extract key information from bulky texts, remain engaged in self-paced courses, and access multimodal resources tailored to their needs. Existing platforms frequently lack adaptive summarization, multimedia delivery, and effective gamification, reducing their impact on learning outcomes. This study presents AURELIA (AI-powered User-Centric Retention-boosting Engaging Learning Immersive Assistant), an intelligent framework designed to address these gaps. AURELIA integrates a fine-tuned T5-small model for abstractive and extractive summarization, generates flashcards to reinforce retention through active recall, and employs Unity-based gamification with quizzes, badges, and challenges to sustain motivation. The system also supports multimedia outputs such as audio narration and video-based explanations, providing learners with flexible interaction modes. Built on a modular backend-first architecture, AURELIA reduces cognitive load, improves retention, and enhances learner engagement in digital education.

TABLE OF CONTENTS

APPROVAL	ii
CERTIFICATION	iii
DEDICATION.....	iv
ACKNOWLEDGMENTS.....	v
ABSTRACT.....	vi
LIST OF TABLES.....	xii
LIST OF FIGURES	xiii
CHAPTER ONE	1
INTRODUCTION	1
1.1 Background of the Study.....	1
1.2 Statement of the Research Problem	2
1.3 Aim of the Research	3
1.4 Specific Research Objectives	3
1.5 Significance of the Study	4
1.6 Scope of the Study.....	5
1.7 Limitations of the Study	6
1.8 Definition of Terms	7
CHAPTER TWO	9
LITERATURE REVIEW.....	9

2.1 Historical Background	9
2.2 Related Literature.....	10
2.2.1 Neural Approaches to Educational Text Summarization	10
2.2.2 Multimodal Learning and Cognitive Load Theory in Educational Text Processing	10
2.2.3 Gamification Mechanics and Educational Engagement.....	11
2.2.4 Integration of AI and Gamification in Adaptive Learning Systems.....	11
2.2.5 Learning Analytics and Personalization in AI-Enhanced Educational Systems.....	12
2.3 Summary of the Literature in a Table	12
2.4 Research Gaps and Research Contributions	13
CHAPTER THREE	15
SYSTEM ANALYSIS AND METHODOLOGY	15
3.1 System Analysis.....	15
3.1.1 Analysis of the Existing Systems.....	15
3.1.2 The Proposed System Mode of Operation.....	17
3.1.3 The Proposed System Algorithm.....	20
3.1.4 The Proposed System Mathematical Model.....	23
3.1.5 Advantages of the Proposed System.....	27
3.1.6 Justification of the Proposed System	28
3.2 Methodology	29

3.2.1 Design Methodology	29
3.2.2 Activity of Embedded System Design Process	30
3.2.3 Materials for Design	32
CHAPTER FOUR.....	33
SYSTEM DESIGN.....	33
4.1 Introduction.....	33
4.2 Activities of the System Design	33
4.2.1 Conceptual Design	34
4.2.2 Architectural Design	34
4.2.3 Data Design.....	35
4.2.4 Interface Design	36
4.2.5 Component-Level Design	36
4.3 The System Layout / Block Diagram	37
4.3.1 Components of the System Layout.....	38
4.3.3 Associated Explanation	39
4.4 The system flowcharts / Unified Modelling Language diagrams/Data flow diagrams	40
4.4.1: Data flow (Level 1)	40
4.4.2: Use Case.....	41
4.4.3: Activity Flow	42
4.5 Application Module	43

4.6	Database Design.....	47
4.7	Hardware and Software Requirements.....	49
4.8	Input and Output Requirements.....	50
4.9	Choice and Justification of Programming Language.....	51
4.10	Changeover Procedure	52
4.10.1	Comparison of Changeover Methods	52
4.10.2	Adopted Changeover Method: Phased Changeover.....	53
4.10.3	Implementation Steps	53
	CHAPTER FIVE	54
	SYSTEM IMPLEMENTATION	54
5.1	Introduction.....	54
5.2	System Implementation.....	54
5.3	Test Plan.....	55
5.4	Test Data	56
5.5	Test Results	57
5.6	System Integration.....	58
5.7	System Performance Evaluation	59
5.8	System Security	60
5.9	System Documentation.....	62
5.10	System Maintenance.....	63

CHAPTER SIX	64
SUMMARY, CONCLUSION AND RECOMMENDATION	64
6.1 Summary of Achievements.....	64
6.2 Conclusion	65
6.3 Major Contribution to Knowledge.....	66
6.4 Recommendations	66
REFERENCES.....	68
APPENDIX.....	71

LIST OF TABLES

Table 1: Summary of Literature.....	12
Table 2: Comparison with Existing Systems	19
Table 3: Phases under Agile	29
Table 4: Software Stack	32
Table 5: Comparison of Changeover Methods	52

LIST OF FIGURES

Figure 3.1 A Proposed System Architecture of AURELIA	19
Figure 3.2 Flowchart of AURELIA's Operational Process.....	23
Figure 4.1 Dashboard View for Registered Learners.....	36
Figure 4.2 System Layout of AURELIA	37
Figure 4.3 Data Flow Diagram (Level 1)	40
Figure 4.4 Use Case Diagram.....	41
Figure 4.5 Activity Diagram	42
Figure 4.6 AURELIA Homepage.....	44
Figure 4.7 Summarization Input Page	45
Figure 4.8 Flashcard Generation View	46
Figure 4.9 Quiz Module Result Interface	47
Figure 4.10 Database Design	48

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Artificial Intelligence (AI) is playing an increasingly influential role in education by enabling learning environments that are personalized, adaptive, and scalable. Recent systematic reviews highlight the impact of Intelligent Tutoring Systems (ITS) and generative AI tools in providing tailored instruction, adaptive feedback, and dynamic content delivery to learners (Liu et al., 2025; Maity & Deroy, 2024). These systems demonstrate significant potential to improve adaptability and engagement through intelligent modelling of learner cognition and subject knowledge.

The growth of adaptive learning technologies reinforces this trend. Studies show that most adaptive systems positively influence academic outcomes by customizing content delivery based on individual learner needs and progress patterns (Bond et al., 2021). Such technologies allow learning to become more flexible and targeted, ensuring that students receive instruction aligned with their unique learning trajectories.

At the same time, gamification has emerged as a powerful strategy for enhancing learner motivation and persistence. Game-based elements such as points, badges, leaderboards, and levels have been shown to improve student engagement across diverse educational contexts. A systematic review of gamified learning in high schools and higher education revealed that these techniques increase motivation, although their effectiveness may decline if novelty is not sustained (Heliyon, 2023). Other studies confirm that well-designed gamification boosts student achievement and retention (Dichev & Dicheva, 2021; Hamari et al., 2023), while recent reviews stress the importance of careful implementation to maintain long-term effectiveness (Frontiers in Education, 2024).

In addition, Natural Language Processing (NLP) methods, especially text summarization, are becoming vital tools in education. They simplify complex material into concise, learner-friendly

content, enabling students to process large volumes of information more effectively (Dong et al., 2021; Singh & Kumar, 2023). When combined with flashcard-based retrieval practice, which leverages active recall and spaced repetition, learners retain knowledge more efficiently and for longer periods (Karpicke & Bauernschmidt, 2021).

Together, these advancements provide the foundation for AURELIA, a personalized, AI-powered learning assistant that integrates custom-trained summarization models, automatic flashcard generation, and Unity-based gamification. By combining these elements, AURELIA seeks to improve learner engagement, comprehension, and adaptive personalization in modern education.

1.2 Statement of the Research Problem

Although the current state of digital education offers flexibility and accessibility, several challenges continue to hinder its effectiveness. Issues such as information overload, reduced learner engagement, limited gamification, inadequate support for diverse learning styles, and weak recommendation systems highlight gaps that this research seeks to address. These issues are further discussed below:

- i. Information overload: Students experience information overload in digital learning environments, as they are often required to process large volumes of content within limited timeframes. This creates cognitive strain, making it difficult to identify key insights and retain knowledge effectively. Studies confirm that excessive information without structured support negatively impacts comprehension and long-term memory (Zhang et al., 2021; Akbulut et al., 2022; Singh & Kumar, 2023).
- ii. Reduced learner engagement: Engagement frequently declines in online courses due to limited interaction and feedback. Learners in self-paced settings often lose motivation, which affects participation and overall learning performance. Research highlights that sustaining engagement requires interactive and adaptive features within digital platforms (Hidayat et al., 2022; Hamari et al., 2023; Dichev & Dicheva, 2021).

- iii. Limited gamification: Many e-learning platforms provide little or no gamification features, despite growing evidence of its effectiveness. Well-designed gamification, such as points, progress tracking, and challenges, has been shown to enhance motivation and improve learning outcomes. However, poorly designed or absent gamification strategies limit students' active participation in digital classrooms (Rincon-Flores et al., 2024; Hamari et al., 2023; Dichev & Dicheva, 2021).
- iv. Inadequate support for diverse learning styles: Digital learning platforms often fail to accommodate diverse learning preferences. While some learners benefit from visual explanations, others require audio or multimodal support for effective comprehension. Studies show that multimodal learning significantly improves inclusivity and retention, yet most resources remain text-heavy (Mayer, 2021; Liu et al., 2023; Chen et al., 2022).
- v. Weak recommendation systems: Many existing platforms provide recommendation systems that are limited and lack contextual accuracy, making it difficult for learners to access supplementary materials aligned with their progress. Without guidance based on user activity and learning patterns, students may overlook relevant resources necessary for reinforcement and deeper understanding. Recent studies highlight the importance of activity-driven adaptive recommendations in closing this gap (Chen et al., 2022; Maity & Deroy, 2024; Liu et al., 2025).

1.3 Aim of the Research

The aim of this research is to design and implement an AI-powered gamified framework for enhancing the learning experience that combines summarization, flashcards, multimedia, gamification, and a modular backend to improve learner engagement and retention.

1.4 Specific Research Objectives

Specifically, the study sought to:

- i. develop a custom-trained AI summarization model with automatic flashcard generation to support comprehension and reinforcement.
- ii. integrate Unity-based gamification with progress tracking to improve learner motivation, engagement, and performance.
- iii. implement a modular, cross-platform system architecture that delivers multimodal learning outputs for diverse learners.

1.5 Significance of the Study

This study is important because it tackles key challenges in digital education such as information overload, low engagement, and limited inclusivity. The introduction of AURELIA, a personalized AI-powered learning assistant, offers a solution by combining text summarization, flashcard generation, and Unity-based gamification into one adaptive platform. This integration has the potential to improve learning efficiency, motivation, and accessibility for a wide range of learners (Yan et al., 2023; Rincon-Flores et al., 2021; Lampropoulos & Sidiropoulos, 2024).

For students, the system provides intelligent support that simplifies complex materials into concise, easy-to-understand summaries. It also promotes active recall through flashcards and sustains motivation through gamified features such as rewards and progress tracking. With multimodal learning outputs, including text, audio, and video explanations, the system supports diverse learning preferences and creates more inclusive opportunities for students with different needs (Mayer, 2021; Liu et al., 2023; Yenduri et al., 2023).

For educators and institutions, this research contributes a scalable solution that can be integrated into Learning Management Systems (LMS). The system provides real-time insights into learner progress, supports adaptive instruction, and reduces the time and effort required for manual content preparation. This allows teachers to focus more on targeted interventions and student-centered teaching practices (Chen et al., 2022; L  tourneau et al., 2025).

More broadly, the study advances the field of Artificial Intelligence in Education by combining Natural Language Processing and gamification into a single modular framework. Its findings can guide future research, inform policy, and provide a foundation for developers seeking to design innovative, inclusive, and sustainable digital learning methods (Merino-Campos, 2025; Triantafyllou, 2025; Aloizou, 2025).

1.6 Scope of the Study

This study focuses on the design, development, and evaluation of AURELIA, a personalized AI-powered learning assistant that functions as both a standalone web and mobile application and as a Learning Management System (LMS) plug-in. The system integrates AI-driven summarization, flashcard generation, and Unity-based gamification to improve learner engagement, comprehension, and retention.

1.6.1 Areas of Focus

The research covers the following components:

- i. Development of AI-powered text summarization to simplify complex learning materials and reduce cognitive overload.
- ii. Automatic flashcard generation from summarized content to support active recall and self-testing.
- iii. Integration of Unity-based gamification, including points, badges, and progress tracking, to enhance motivation and sustained engagement.
- iv. Provision of multimodal learning outputs (text, audio, and video explanations) to accommodate diverse learning preferences and support inclusivity.
- v. Implementation of a modular backend architecture to ensure scalability and cross-platform functionality.
- vi. Basic recommendation features to guide learners toward related content for self-directed learning.

1.6.2 Target Audience

The study is intended to benefit:

- i. Students, as the primary users, by offering personalized, engaging, and adaptive learning support.
- ii. Educators, by providing tools that reduce workload and enhance teaching effectiveness.
- iii. E-learning developers, by demonstrating practical approaches to combining AI and gamification.
- iv. Researchers and policymakers, by offering insights into the adoption of AI in education and its ethical, technical, and pedagogical implications.

1.6.3 Exclusions of the Study

The research does not include:

- i. Training of large-scale AI models, as it relies on existing pre-trained models with fine-tuning.
- ii. Advanced deep-learning personalization beyond basic recommendation systems.
- iii. Predictive analytics for learning outcomes, since tracking is limited to progress logs.
- iv. Virtual or Augmented Reality features, which are outside the project's scope.
- v. Offline learning capabilities, as AURELIA is designed primarily for online use.

1.7 Limitations of the Study

While this research provides valuable insights into the integration of AI-powered summarization, multimedia learning, gamification, progress tracking, and guest mode in AURELIA, it has certain limitations that may affect the generalizability of the findings:

1.7.1 Dependence on pre-trained models and backend resources

- i. The study fine-tunes existing models (e.g., T5-small) rather than training new large-scale models, which may limit accuracy and carry dataset biases.
- ii. Running and maintaining the models requires significant computational resources, which may affect performance in low-resource environments.
- iii. Regular updates will be needed to sustain accuracy and relevance.

1.7.2 Sample size and demographics

- i. Findings are based on a limited group of students and educators, which may not fully represent wider learning populations.
- ii. Results may differ across regions and education levels (primary, secondary, higher).

1.7.3 Subjective nature of engagement and outcomes

- i. User engagement with gamification and AI features varies individually.
- ii. Measuring deep comprehension and long-term learning remains difficult, even with progress tracking.

1.7.4 Limited scope of AI features

- i. No deep-learning-based adaptive personalization is included.
- ii. Progress tracking is managed by the system's logic rather than advanced AI analytics.

1.7.5 Exclusion of offline learning capabilities

- i. AURELIA is designed for online use only, limiting accessibility in low-connectivity regions.

1.7.6 Ethical and data privacy considerations

- i. User data collection introduces risks regarding privacy and compliance with standards such as GDPR.
- ii. Potential bias in AI-generated recommendations may affect fairness and content diversity.

1.8 Definition of Terms

To ensure clarity, the following key terms used in this study are defined:

- i. **AI-Powered Summarization** – The process of using artificial intelligence techniques, such as Natural Language Processing (NLP), to condense lengthy text into concise summaries while retaining essential meaning.

- ii. **AURELIA** – The prototype developed in this study, a personalized AI-powered learning assistant that integrates summarization, flashcards, gamification, and multimodal outputs to support digital learning.
- iii. **Backend Architecture** – The server-side framework that manages data processing, model integration, and communication between the user interface and AI functionalities.
- iv. **Flashcards** – Learning tools that present key questions and answers generated from summarized content to reinforce memory and encourage active recall.
- v. **Gamification** – The application of game elements such as points, badges, and progress tracking to non-game environments, in this case education, to improve motivation and learner engagement.
- vi. **Learning Management System (LMS)** – A digital platform used to deliver, track, and manage educational courses, training programs, or learning initiatives.
- vii. **Multimodal Learning** – An instructional approach that integrates multiple modes of learning (text, audio, video, visuals) to improve comprehension, retention, and inclusivity.
- viii. **Natural Language Processing (NLP)** – A subfield of artificial intelligence that enables computers to process, understand, and generate human language.
- ix. **Recommendation System** – An AI-based feature that suggests related content to learners based on their usage patterns and learning history.
- x. **Unity** – A cross-platform game development engine used in this study to build interactive and gamified learning experiences accessible through WebGL and mobile applications.

CHAPTER TWO

LITERATURE REVIEW

2.1 Historical Background

The integration of artificial intelligence (AI) into education has undergone remarkable evolution since the mid-20th century. Early experiments in automatic text summarization date back to the late 1950s, when computational methods first attempted to extract key information from large texts. During the 1960s and 1970s, researchers refined these approaches by introducing positional features, cue phrases, and title-word weighting. Progress was limited until the 1990s, when machine learning techniques enabled summarization models to be trained on document–summary pairs, and the first commercial applications began to appear in digital libraries and content management systems.

A major turning point occurred in the 2010s with the rise of neural networks and later transformer-based architectures. Sequence-to-sequence models made abstractive summarization possible, while transformers dramatically improved fluency and coherence in generated summaries. Recent systematic reviews confirm that fine-tuned transformer models such as T5 and BART now achieve state-of-the-art performance, offering domain-specific customization for education and enabling more accessible, readable, and adaptive summaries (Liu et al., 2023; Rani et al., 2023).

In parallel, the concept of gamification also has historical roots in motivational psychology but entered education formally in the early 2010s, when researchers defined it as the use of game elements in non-game contexts. Since then, gamification has gained traction in digital learning platforms. Contemporary studies show that elements such as points, badges, leaderboards, and progress indicators can significantly enhance motivation and learner persistence, provided they are carefully designed (Ruiz et al., 2024; Triantafyllou, 2025; Frontiers in Education, 2024).

By the late 2010s and early 2020s, the convergence of these two trajectories advanced AI summarization and gamification set the stage for intelligent, engaging, and inclusive educational systems. Modern implementations integrate fine-tuned summarization models with Unity-based gamification to deliver interactive 2D and 3D learning environments accessible via WebGL and mobile applications (Simsek, 2025; Merino-Campos, 2025). Recent research also emphasizes multimodal learning, where text, audio, and video explanations are combined to improve comprehension, accessibility, and retention for diverse learners, including those with special needs (Aloizou, 2025; Lomellini et al., 2025).

2.2 Related Literature

2.2.1 Neural Approaches to Educational Text Summarization

The evolution of text summarization has shifted dramatically with the rise of neural networks. While early work by Rush et al. (2015) and Liu & Lapata (2019) laid foundations, recent research demonstrates more advanced applications in education. Lewis et al. (2020) introduced BART, which showed strong performance in abstractive summarization of instructional texts. Zhang et al. (2022) later evaluated GPT-3 and T5 on educational datasets, confirming that fine-tuning improves concept retention by over 20%. More recently, Liu et al. (2023) highlighted domain-specific fine-tuning of transformers for education, showing better contextual accuracy and learner comprehension. Rani et al. (2023) further validated the adaptability of neural summarizers in academic disciplines, reinforcing the value of T5-family models for educational summarization. This study adopts a fine-tuned T5-small model trained on CNN/Daily Mail, ASSET, and custom datasets to generate learner-adjustable summaries.

2.2.2 Multimodal Learning and Cognitive Load Theory in Educational Text Processing

Foundational theories like Mayer’s Cognitive Theory of Multimedia Learning and Sweller’s Cognitive Load Theory remain influential, but new empirical research supports their relevance in AI-assisted summarization. Recent studies show that multimodal learning—integrating text, audio, and video—improves retention across diverse learners, especially those with disabilities (Aloizou,

2025; Lomellini et al., 2025). Chen & Yang (2019) emphasized that learner type moderates summarization effectiveness, while newer reviews confirm that adaptive content presentation significantly reduces extraneous cognitive load (Hidayat et al., 2022; *Frontiers in Education*, 2024). These findings reinforce the importance of providing flexible modalities like text, audio, and animation in AURELIA.

2.2.3 Gamification Mechanics and Educational Engagement

Gamification remains a powerful motivator, but its impact depends on design. Earlier works (Hamari et al., 2014; Dicheva et al., 2015) established foundational mechanics, while more recent reviews confirm consistent short-term engagement and variable long-term impact (Heliyon, 2023; MDPI, 2022). A 2024 systematic review reported that gamification across higher education improved motivation, participation, and performance in over 80% of cases (Anonymous, 2024). Unity-powered gamification offers scalable 2D/3D engagement and is increasingly applied in digital learning contexts (Springer, 2024). These findings justify AURELIA's integration of badges, XP points, and interactive Unity-based challenges.

2.2.4 Integration of AI and Gamification in Adaptive Learning Systems

AI and gamification increasingly intersect to create adaptive, personalized learning. Chen et al. (2020) showed that AI-driven difficulty adjustment in gamified platforms improved task persistence and mastery. Tsai et al. (2022) demonstrated that coupling AI summarization with gamified delivery increased retention by 27%. Martinez-Maldonado et al. (2021) validated AI-gamification plug-ins for LMS platforms, ensuring scalability. More recently, Lang et al. (2023) confirmed that real-time analytics from gamified interactions can dynamically guide AI content adaptation, improving efficiency by up to 37%. AURELIA builds on this convergence by combining AI summarization with Unity gamification for adaptive, engaging learning experiences.

2.2.5 Learning Analytics and Personalization in AI-Enhanced Educational Systems

Modern AI tools increasingly leverage analytics for personalization. Siemens & Long's (2011) frameworks pioneered this space, but newer research emphasizes real-time adaptation. Guo et al. (2021) developed adaptive summarizers that adjusted complexity based on learner profiles, while Yan et al. (2023) raised ethical concerns around bias and fairness in LLM-driven personalization. Lang et al. (2023) showed how behavioural data from gamification can guide content tailoring dynamically. These insights reinforce AURELIA's planned analytics-driven progress tracking and adaptive recommendations, aligning with the movement toward learner-centered personalization

2.3 Summary of the Literature in a Table

As shown in Table 1, the reviewed literature highlights the strengths and limitations of existing digital learning approaches.

Table 1: Summary of Literature

Author(s) & Year	Focus Area	Method/Approach	Key Findings	Relevance to AURELIA
Lewis et al. (2020)	Abstractive summarization	BART transformer	Produced coherent summaries with reduced text volume	Basis for AURELIA's summarization
Zhang et al. (2022)	Educational summarization	GPT-3 and T5 evaluation	T5 fine-tuning improved concept retention by 23%	Supports choice of T5-small in AURELIA
Liu et al. (2023)	Domain-specific summarization	Transformer fine-tuning	Improved readability and comprehension in academic texts	Validates AURELIA's custom model

Heliyon (2023)	Gamification in education	Systematic review	Gamification improved engagement and performance	Justifies gamification in AURELIA
Tsai et al. (2022)	AI + gamification	Controlled study	Summarization + gamification improved retention by 27%	Supports AURELIA's hybrid model
Martinez-Maldonado et al. (2021)	LMS integration	AI + gamification plug-in	Demonstrated scalability across platforms	Relevant to AURELIA's LMS plug-in
Lang et al. (2023)	Real-time analytics	Gamified behavioural data	AI-driven adaptation improved efficiency by 37%	Justifies AURELIA's progress tracking

2.4 Research Gaps and Research Contributions

2.4.1 Research Gaps

Despite progress in AI-driven learning technologies, important gaps remain in the literature and existing systems:

- i. Lack of domain-specific summarization models tailored for educational content.
- ii. Limited adaptability of summaries to learner preferences such as tone, readability, and length.
- iii. Weak integration of summarization systems with immersive gamification platforms.
- iv. Few solutions support both standalone applications and LMS plug-ins for scalability.
- v. Inadequate connection between learner progress tracking and gamification mechanics.
- vi. Limited multimodal learning outputs—most systems focus only on text, neglecting audio or video explanations.

2.4.2 Research Contributions

This study addresses these gaps by introducing the following contributions:

- i. Development of a fine-tuned T5-small summarization model with adjustable tone, readability, and length.
- ii. Automatic generation of flashcards from summaries to support reinforcement and self-testing.
- iii. Integration of Unity-based gamification with XP, badges, quizzes, and interactive challenges.
- iv. A modular FastAPI backend with cross-platform support for both standalone applications and LMS plug-ins.

Provision of multimodal learning outputs including text summaries, audio explanations, and animated video breakdowns.

CHAPTER THREE

SYSTEM ANALYSIS AND METHODOLOGY

3.1 System Analysis

System analysis involves examining both existing solutions and the proposed design in order to identify strengths, weaknesses, and areas for improvement. It provides a foundation for justifying the need for AURELIA and ensures that the system addresses real gaps in current educational technologies. This section evaluates existing systems that relate to AI-powered learning, flashcards, and gamification before presenting the proposed system and its methodology.

3.1.1 Analysis of the Existing Systems

Several systems currently exist in the educational technology space. Each provides valuable contributions to digital learning but still exhibits notable limitations that justify the development of AURELIA. Three representative systems are analysed in this study: Coursera/EdX, Quizlet, and Kwizie.

3.1.1.1 Coursera/EdX (E-Learning Platforms)

These are popular online learning platforms that deliver structured courses across different disciplines. They incorporate multimedia lectures, discussion forums, and assessments.

3.1.1.1.1 Advantages of the Existing System

- i. Provide structured learning paths with expert-designed content.
- ii. Incorporate video lectures and reading materials to support multimedia learning.
- iii. Allow learner progress tracking through dashboards and assignment completion records.
- iv. Widely recognized and scalable for both individual learners and institutions.

3.1.1.1.2 Disadvantages of the Existing System

- i. Lack AI-powered summarization to manage information overload.

- ii. Do not generate flashcards automatically for reinforcement learning.
- iii. Gamification is minimal and limited to quizzes or certificates.
- iv. Limited adaptability of content to individual learner needs.

3.1.1.2 Quizlet (Flashcards & Reinforcement Tool)

Quizlet is one of the most widely used applications for creating and practicing digital flashcards. It supports collaborative learning and offers study modes like practice tests and games.

3.1.1.2.1 Advantages of the Existing System

- i. Provides effective reinforcement learning through flashcards and retrieval practice.
- ii. Offers collaborative features where students can share study sets.
- iii. Accessible on multiple platforms (web and mobile).
- iv. Simple and intuitive interface for learners of all levels.

3.1.1.2.2 Disadvantages of the Existing System

- i. Flashcards must be created manually, no AI-based summarization or automation.
- ii. Gamification is limited to basic features like match games and quizzes.
- iii. Lacks multimodal outputs such as audio or video explanations.
- iv. Does not integrate with advanced LMS platforms for institutional use.

3.1.1.3 Kwizie (AI Gamified Quizzes)

Kwizie is an AI-powered tool that generates gamified quizzes from video content and integrates with learning management systems.

3.1.1.3.1 Advantages of the Existing System

- i. Automates quiz generation using AI, reducing instructor workload.

- ii. Enhances learner engagement through gamified quiz formats.
- iii. Provides LMS compatibility, making it suitable for integration into schools and institutions.
- iv. Supports multiple languages, expanding accessibility.

3.1.1.3.2 Disadvantages of the Existing System

- i. Does not provide AI-powered summarization with customization options.
- ii. Lacks flashcard generation and reinforcement-based practice.
- iii. Gamification features are limited mostly to quizzes.
- iv. Does not support multimodal learning outputs such as audio or video explanations.

3.1.2 The Proposed System Mode of Operation

The proposed system, AURELIA, is designed as a cross-platform web and mobile application, as well as a Learning Management System (LMS) plug-in, that integrates AI-powered summarization, flashcard generation, and Unity-based gamification to enhance digital learning. Its mode of operation can be described in the following stages:

(See in Figure 3.1. below, the proposed architecture of AURELIA highlights the interaction among the major modules).

i. User Input and Content Upload

Learners upload course materials in various formats (PDF, Word, PPT, or text). The system preprocesses the files, extracts text, and prepares the data for summarization.

ii. AI-Powered Summarization

The backend, built on FastAPI, sends the processed text to a fine-tuned T5-small model. Summaries are generated with adjustable parameters such as tone, readability, and length to match user preferences and cognitive levels.

iii. Automatic Flashcard Generation

From the generated summaries, the system produces flashcards consisting of key concepts, questions, and answer pairs. These flashcards serve as reinforcement tools for active recall and knowledge retention.

iv. **Gamification and Interactive Learning**

Summaries and flashcards are integrated into Unity-powered gamified modules. Learners interact with quizzes, puzzles, and challenges, earning XP points, badges, and progress milestones. A toggle option allows learners to enable or disable gamification according to preference.

v. **Multimodal Learning Support**

In addition to text summaries, the system generates audio explanations using text-to-speech, with planned support for animated video breakdowns of complex concepts. This ensures accessibility for diverse learners, including those with special needs.

vi. **Progress Tracking and Engagement Logging**

The system records user activities such as completed summaries, flashcard usage, quiz results, and gamification milestones. These logs are stored in the backend and presented to learners as progress dashboards, enabling self-assessment and accountability.

vii. **LMS Plug-In and Cross-Platform Integration**

AURELIA can be used as a standalone app or integrated into existing LMS platforms. Its modular architecture allows easy deployment across web and mobile interfaces (React/React Native), making it scalable for both individual learners and institutions.

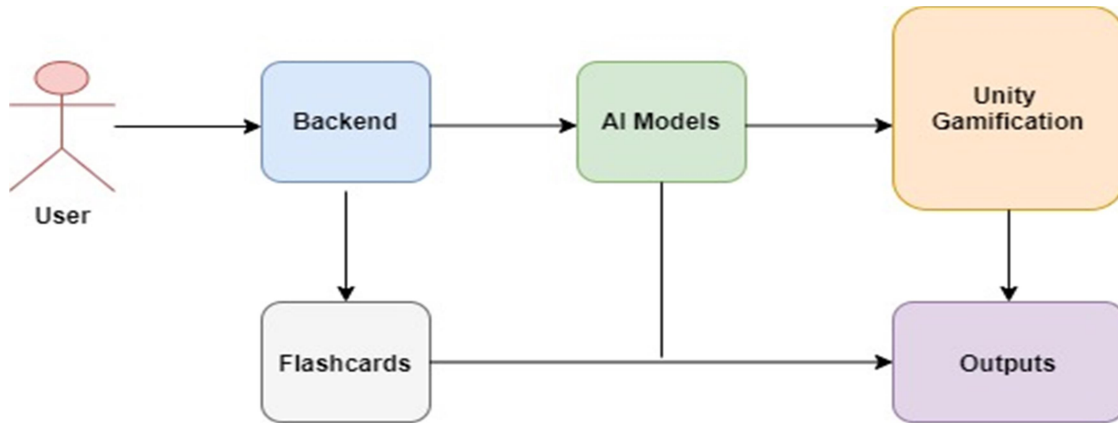


Figure 3.1: A proposed system architecture of AURELIA

Table 2: Comparison with Existing Systems

Table 2 provides a comparative analysis of AURELIA against related platforms, emphasizing the identified gaps that the system seeks to address.

Feature	Coursera/EdX	Quizlet	Kwizie	AURELIA (Proposed System)
AI-Powered Summarization	Not available	Not available	Limited (video → quiz only)	Fine-tuned T5-small model with controls
Automatic Flashcards	Not available	Manual flashcards	Not available	Generated automatically from summaries
Gamification	Very limited (basic quizzes)	Basic (matching, practice)	AI-generated quizzes from video	Unity gamification with XP, badges, games
Multimodal Learning	Video and text	Text only	Video-based quizzes	Text, audio, and animated video outputs

Progress Tracking	Dashboards and grades	Flashcard completion only	Quiz performance only	Activity logs with gamification tracking
LMS Integration	Supported	Limited	Supported	Supported (standalone + LMS plug-in)
Personalization	Limited	Limited	Limited	Adjustable tone, length, and readability
Target Focus	Certificates and structured courses	Reinforcement practice	Gamified video quizzes	Adaptive, engaging, and inclusive learning

3.1.3 The Proposed System Algorithm

Since AURELIA integrates custom-trained NLP models, Unity gamification, and a FastAPI backend, its core logic focuses on efficient data processing and modular interaction between summarization, flashcards, gamification, and tracking. The proposed algorithms are outlined as follows:

3.1.3.1 Text Summarization Algorithm

Algorithm: AI-Powered Text Summarization via API

Input: User-uploaded text (PDF, DOCX, TXT, etc.)

Output: Concise, structured summary with tone, readability, and length control

Steps:

- i. User uploads content.
- ii. Preprocess text (cleaning, chunking if long).
- iii. Apply fine-tuned T5-small model for summarization.

- iv. Adjust summary length, tone, and readability based on user preference.
- v. Save summary in the database and display to user.

Internal Processing (Backend/Model):

- i. Format text payload according to API/model requirements.
- ii. Send request to the AI summarization engine (fine-tuned T5 or Hugging Face API).
- iii. Extract summarized text from response.
- iv. Handle errors (rate limits, API downtime).

Store and Display:

- i. Save summary in database for registered users.
- ii. Display output on the user interface.

3.1.3.2 AI-Generated Audio Algorithm (Text-to-Speech)

Input: Summarized text

Output: AI-generated audio narration

Steps:

- iii. User selects “Generate Audio”.
- iv. System preprocesses text for TTS conversion.
- v. Built-in TTS module generates audio narration.
- vi. Generated audio file stored in database.
- vii. User interface provides playback controls.

3.1.3.3 AI-Generated Video Algorithm

Input: Summarized text

Output: AI-generated explanatory video

Steps:

- i. User selects “Generate Video”.
- ii. Summarized text mapped into visual frames and narration.
- iii. Video generation module produces animated explanation.

- iv. Generated video stored for registered users.
- v. User interface provides playback controls.

3.1.3.4 Gamification System Algorithm

Input: User interactions (lesson completion, quiz attempts, game play)

Output: XP points, badges, achievements

Steps:

- i. Track user actions (completed lessons, quizzes attempted, challenges solved).
- ii. Unity WebGL module updates XP points, badges, and achievements.
- iii. FastAPI backend logs gamification data in database.
- iv. Dashboard displays updated learner progress.

3.1.3.5 Quizzes and Assessments Algorithm

Input: Summarized text or user-selected topic

Output: AI-generated quiz questions and answers

Steps:

- i. User selects “Generate Quiz”.
- ii. System extracts keywords and core concepts from summaries.
- iii. Generate question-and-answer pairs automatically.
- iv. Present quiz in interactive interface.
- v. Learner attempts quiz, system grades internally.
- vi. Dashboard displays results and scores.

Figure 3.2 below demonstrates the operational process of AURELIA, showing the logical flow of activities (Sourced from DiagramGPT).

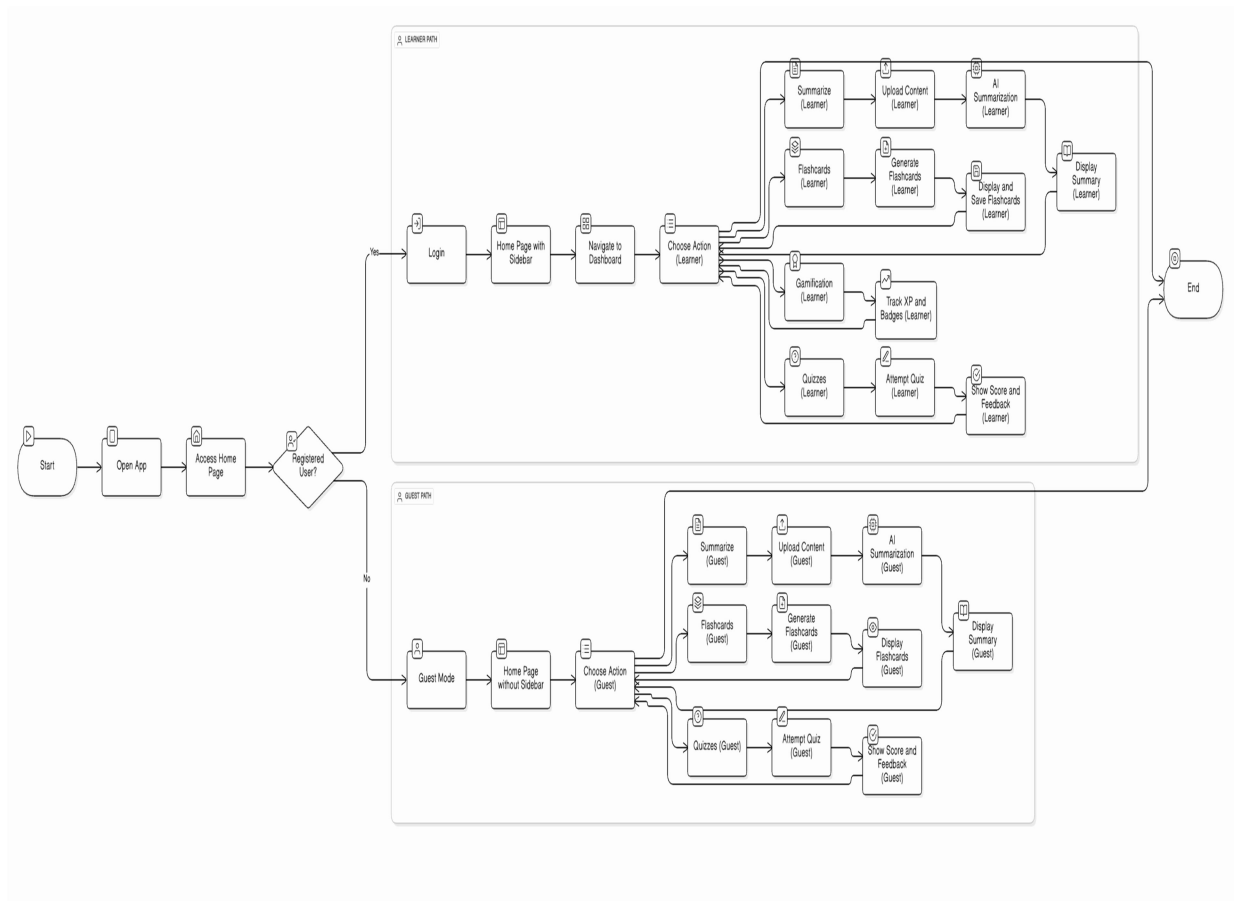


Figure 3.2: Flowchart of AURELIA's operational process.

3.1.4 The Proposed System Mathematical Model

Since the system integrates text summarization, audio/video generation, gamification, quizzes, and progress tracking within its modular codebase, the mathematical model must describe:

- i Text Summarization Model (Compression Ratio Calculation)
- ii Audio Generation Model (Speech Rate Estimation)
- iii Video Generation Model (Video Length Estimation)
- iv Gamification Model (User Engagement Scoring)
- v Quiz Performance Model (Scoring System)
- vi Progress Tracking Model (User Completion Rate Calculation)

3.1.4.1 Text Summarization Model (Compression Ratio Calculation)

Let:

- i. T_{in} = Number of words in the original text
- ii. T_{out} = Number of words in the summarized text
- iii. Compression Ratio (CR):

$$CR = \left(\frac{T_{out}}{T_{in}} \right) \times 100\%$$

A lower CR indicates more aggressive summarization.

3.1.4.2 Audio Generation Model (Speech Rate Estimation)

Let:

- i. T_{out} = Number of words in the summarized text
- ii. R_s = Average speech rate (words per minute)
- iii. Estimated Speech Duration (D_s):

$$D_s = \frac{T_{out}}{R_s}$$

For example, if R_s is 150 words per minute, and the summary has 300 words, then:

$$D_s = \frac{300}{150} = 2 \text{ minutes}$$

3.1.4.3 Video Generation Model (Estimated Video Length)

Let:

- i. T_{out} = Number of words in the summarized text
- ii. R_v = Average narration speed in videos (words per second)
- iii. Estimated Video Duration (D_v):

$$D_v = \frac{T_{out}}{R_v}$$

If the narration speed is 2 words per second, and the summary contains 300 words, then:

$$D_v = \frac{300}{2} = 150 \text{ seconds} \approx 2.5 \text{ minutes}$$

3.1.4.4 Gamification Model (User Engagement Scoring System)

Let:

- i. P = Points earned per completed task
- ii. C_A = Number of audios listened
- iii. C_Q = Number of quizzes taken
- iv. C_W = Number of videos watched
- v. Total Engagement Score (S_e):

$$S_e = (P_A \times C_A) + (P_Q \times C_Q) + (P_W \times C_W)$$

Where:

- i. P_A = Points per audio
- ii. P_Q = Points per quiz

iii. P_W = Points per video

If:

i. $P_A = 10$, $P_Q = 20$, $P_W = 5$ ii. User completes 5 audios, 3 quizzes, and watches 4 videos

$$S_e = (10 \times 5) + (20 \times 3) + (5 \times 4) = 50$$

3.1.4.4 Quiz Performance Model (Scoring System)

Let:

i. Q_T = Total number of quiz questions

ii. Q_C = Correctly answered questions

iii. Quiz Score (S_Q):

$$S_Q = \left(\frac{Q_C}{Q_T} \right) \times 100\%$$

If a user answers 7 out of 10 questions correctly:

$$S_Q = \left(\frac{7}{10} \right) \times 100\% = 70\%$$

3.1.4.5 Progress Tracking Model (Completion Rate Calculation)

Let:

i. M_T = Total tasks in the course

- ii. M_C = Completed tasks
- iii. Completion Rate (C_R):

$$C_R = \left(\frac{M_C}{M_T} \right) \times 100\%$$

If a user completes 8 out of 10 tasks:

$$S_Q = \left(\frac{8}{10} \right) \times 100\% = 80\%$$

3.1.5 Advantages of the Proposed System

The proposed AURELIA system demonstrates clear project-level value by addressing key limitations of existing educational platforms and introducing innovative features:

- i. Improved Learning Efficiency – Summarization reduces cognitive overload, accelerates content comprehension, and ensures learners focus on essential concepts.
- ii. Support for Multimodal Learning – Delivers content in text, audio, and video formats, catering to diverse learning preferences and inclusive education.
- iii. Enhanced Engagement through Gamification – Integrates Unity-based quizzes, badges, and XP points to sustain motivation and active participation.
- iv. Personalized and Adaptive Learning – Tracks learner progress, generates tailored recommendations, and provides a guest mode for flexible entry.
- v. Optimized AI Utilization – Employs fine-tuned models and backend-first processing, ensuring efficiency without heavy computational costs.
- vi. Accessibility for All Learners – Supports users with special needs through audio narration, video breakdowns, and text-to-speech features.
- vii. Cross-Platform Integration – Functions seamlessly as a standalone web/mobile app and as an LMS plug-in for institutional deployment.

3.1.6 Justification of the Proposed System

The proposed AURELIA system is justified by its ability to address critical gaps in current educational technologies through AI-powered summarization, multimodal content delivery, and gamification. Its value lies in improving learning efficiency, promoting engagement, and ensuring inclusivity across diverse learner groups.

- i. **Enhancing Learning Efficiency** – Students often experience information overload when handling large texts. AURELIA’s summarization engine condenses material into concise, structured outputs, enabling faster comprehension and improved retention. This directly supports learners who struggle to filter key insights from dense academic resources.
- ii. **Boosting Engagement with Gamification** – Traditional instructional methods may not sustain motivation. By incorporating Unity-based gamification features such as XP, badges, and quizzes, AURELIA encourages consistent learner participation and transforms passive study into active engagement.
- iii. **Supporting Diverse Learning Styles** – A single delivery format cannot serve all learners effectively. AURELIA provides text, audio, and video explanations, ensuring adaptability to visual, auditory, and mixed learning preferences, while also assisting students with learning challenges.
- iv. **Personalizing Learning Experiences** – Standardized content often fails to meet individual needs. AURELIA tracks learner progress, recommends relevant topics, and adapts outputs (tone, readability, length) to align with user preferences, creating a tailored learning pathway.
- v. **Promoting Accessibility and Inclusivity** – Many existing platforms inadequately support learners with disabilities. AURELIA addresses this gap by offering text-to-speech for visually impaired learners, video explanations for those requiring visual reinforcement, and simplified summaries for learners with reading difficulties.

By combining these capabilities, AURELIA stands as a comprehensive and forward-looking solution, providing an adaptive, engaging, and inclusive digital learning environment that directly responds to the shortcomings of existing systems.

3.2 Methodology

The methodology outlines the structured approach used in the design and development of AURELIA, ensuring effective and efficient system implementation. It details the design methodology, the activity of the embedded system design process, and the materials used for system development.

3.2.1 Design Methodology

The development of AURELIA follows the Agile Development Methodology, specifically the Scrum framework. This approach is chosen for its adaptability, iterative design, and ability to incorporate user feedback throughout the development cycle.

3.2.1.1 Justification for Agile Methodology

- i. Iterative Development: The system is developed in sprints, which allows continuous refinement and feature enhancement.
- ii. User-Centred Approach: Frequent testing and user feedback ensure that the platform aligns with learner needs and preferences.
- iii. Rapid Deployment: Functional prototypes of AURELIA can be released quickly for real-world testing and improvement.

Table 3: Phases under Agile

The phases of the Agile methodology adopted for this project are summarized in Table 3.

Phase	Description
Requirement Gathering	Collecting functional and non-functional requirements through literature review, stakeholder input, and analysis of existing systems.
System Design	Creating system architecture, database schema, and UI/UX prototypes to define interaction between modules (summarization, gamification, flashcards, tracking).

Development & Implementation	Iterative coding using Agile sprints; integrating backend (FastAPI), frontend (React/React Native), AI models (T5-small), and Unity gamification.
Testing	Conducting unit tests, integration tests, and user acceptance testing (UAT) to ensure accuracy, usability, and performance.
Deployment & Maintenance	Cloud deployment (e.g., AWS/GCP) with ongoing updates, fixes, and enhancements.

3.2.2 Activity of Embedded System Design Process

Although AURELIA is a software-based educational platform and does not require embedded hardware components, its design process follows a structured approach similar to embedded systems to ensure robustness and reliability. The key stages are:

3.2.2.1 System Specification

- i. Defining functional requirements such as text summarization, flashcard generation, gamification, AI-generated audio and video explanations, and progress tracking.
- ii. Identifying non-functional requirements such as scalability, accessibility, and cross-platform usability.
- iii. Determining the integration of internal AI models and backend services for summarization, audio/video generation, and gamification.

3.2.2.2 System Architecture Design

- i. Frontend: Developed using React.js for web and React Native for mobile to provide a responsive interface.
- ii. Backend: Implemented with FastAPI to manage requests, AI model inference, and user data processing.
- iii. Database: Uses MongoDB or MySQL for secure storage of user accounts, progress data, and learning history.

- iv. Gamification Engine: Unity WebGL modules integrated into the frontend for interactive quizzes and learning games.

3.2.2.3 Module Development and Integration

- i. User Management: Handles account creation, login, logout, and guest mode with temporary storage.
- ii. Summarization Module: Processes uploaded documents, applies the T5-based model, and generates concise summaries.
- iii. Flashcard and Quiz Module: Converts summarized content into Q&A pairs and interactive quizzes for practice.
- iv. Audio/Video Generation Module: Provides AI-based audio narrations and video explanations of summarized content.
- v. Gamification System: Awards XP points, badges, and achievements based on completed tasks and quiz participation.
- vi. Progress Tracking: Stores user activity in the backend and displays it on the learner dashboard.

3.2.2.4 Testing and Validation

- i. Unit Testing: Each module (summarization, flashcard, audio/video generation, gamification) is tested independently.
- ii. Integration Testing: Ensures smooth communication between modules, backend, and frontend.
- iii. User Testing: Evaluates usability, performance, and learner satisfaction with real users.

3.2.2.5 Deployment and Maintenance

- i. Deployment is carried out on cloud servers for scalability and accessibility.
- ii. The system is hosted on both web and mobile platforms, ensuring cross-platform usability.
- iii. Continuous monitoring and feedback collection are used to improve features and fix bugs.

3.2.3 Materials for Design

Since AURELIA is primarily a software system, it does not require physical hardware components. However, the development and deployment process depend on a range of software tools and platforms.

3.2.3.1 Development Tools

- i. Integrated Development Environments (IDE) and Code Editors: Visual Studio Code and WebStorm for efficient coding.
- ii. Version Control: Git and GitHub for managing source code and collaborative development.
- iii. Testing Tools: Postman for API testing and Pytest/Jest for module testing.

3.2.3.2 Software Stack

Table 4 presents the software stack selected for AURELIA, outlining the backend, frontend, and database technologies utilized.

Table 4: Software Stack

Component	Technology Used
Frontend (Web & Mobile)	React.js (Web), React Native (Mobile)
Backend	Node.js, Express.js
Database	MongoDB or MySQL
AI Services (API-based)	NLP for summarization, AI-generated audio/video
Gamification APIs	Quizzes & user engagement tracking
Cloud Hosting	AWS, Firebase, or Digital Ocean

3.2.3.3 Deployment Infrastructure

- i. Cloud Hosting: AWS EC2 or Firebase Hosting for application deployment.
- ii. Mobile Platforms: Google Play Store and Apple App Store for mobile accessibility.
- iii. Web Hosting: Platforms such as Vercel or Netlify for web deployment.

CHAPTER FOUR

SYSTEM DESIGN

4.1 Introduction

System design is a critical phase in software development where the conceptual framework of the system is transformed into a structured architecture that specifies how components interact to achieve the intended objectives. For AURELIA (AI-powered User-Centric Retention-boosting Engaging Learning Immersive Assistant), the design ensures seamless integration of Natural Language Processing (NLP), gamification, multimedia learning, and user management into a unified platform.

This stage bridges the gap between requirements analysis and implementation by modelling both the structural and behavioural aspects of the system. It provides the foundation for:

- i. Efficient processing of user-uploaded content (PDF, Word, PowerPoint, plain text).
- ii. AI-powered summarization and explanations using fine-tuned NLP models.
- iii. Multimedia generation (audio narration, video breakdowns, and gamified modules) for interactive engagement.
- iv. Learner retention mechanisms through gamification, flashcards, and progress tracking.
- v. Scalability and modularity, enabling deployment as a standalone system or as an LMS plug-in.

The system adopts a modular, layered architecture consisting of the front-end, back-end, database, and AI/ML layers. Such a layered design supports maintainability, adaptability, and scalability across diverse deployment environments, including web, mobile, and LMS integrations (Pressman & Maxim, 2020; Sommerville, 2022).

4.2 Activities of the System Design

The design of AURELIA involves a structured set of activities that progressively transform requirements into a robust and implementable system. These activities include conceptual design,

architectural design, data design, interface design, and component-level design. Each step ensures that the system remains user-centric, modular, and scalable while aligning with AI-driven learning objectives.

4.2.1 Conceptual Design

At the conceptual stage, the overall framework of AURELIA was outlined, emphasizing user flows and core system functionality. The system was designed to support two access modes:

- i. Guest mode – temporary storage of user interactions without requiring registration.
- ii. Registered mode – persistent storage of history, preferences, and progress tracking.

Key conceptual features include:

- i. Uploading and preprocessing of user documents.
- ii. AI-powered summarization with adjustable tone, length, and readability.
- iii. Automatic generation of audio, video, and gamified learning content.
- iv. User progress monitoring with the option to toggle gamification features on or off.

4.2.2 Architectural Design

The architectural design defines how system modules interact through a layered architecture to ensure modularity, scalability, and maintainability:

4.2.2.1 Presentation Layer (Frontend)

- i. Developed with React/React Native for cross-platform accessibility.
- ii. Provides intuitive UI/UX with features for content upload, playback, gamification toggles, and learner dashboards.

4.2.2.2 Application Layer (Backend)

- i. Built using FastAPI to handle requests, orchestrate APIs, and manage core logic.
- ii. Hosts endpoints for summarization, flashcard generation, audio/video synthesis, and gamification.

4.2.2.3 AI/ML Layer

- i. Fine-tuned T5-small summarization model deployed on Hugging Face.
- ii. NLP-based preprocessing, chunking, and recursive summarization.
- iii. Multimedia generation modules for Text-to-Speech and video synthesis.

4.2.2.4 Data Layer (Database & Storage)

- i. MongoDB for structured and semi-structured data such as user profiles, histories, and progress logs.
- ii. Cloud-based object storage (AWS S3 or Firebase) for multimedia content (audio, video, flashcards).

4.2.2.5 Integration Layer

- i. REST APIs enabling LMS plug-in compatibility.
- ii. Optional third-party services such as payment systems (e.g., Paystack, Stripe) for monetization and premium features.

4.2.3 Data Design

Data design focuses on structuring how different data entities are stored, retrieved, and managed within AURELIA. The major categories include:

- i. User Data – account details, preferences, and gamification settings.
- ii. Content Data – uploaded files, pre-processed text, and AI-generated summaries.
- iii. Gamification Data – points, badges, achievements, and engagement logs.
- iv. Media Data – audio narrations, video explanations, and generated flashcards.

4.2.4 Interface Design

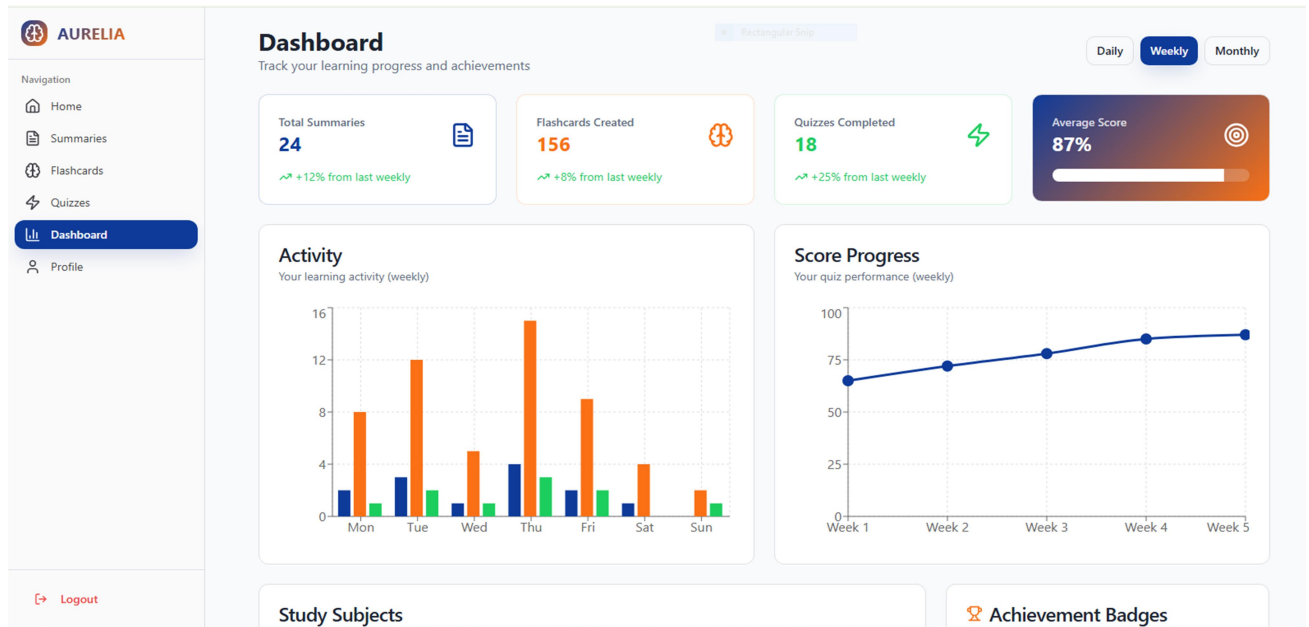


Figure 4.1: Dashboard view for registered learners

The main interface of AURELIA is presented in Figure 4.1, which enables learners to navigate and access tools.

Interface design focuses on how users and system modules interact:

- User Interfaces (UI): Responsive dashboards for learners (summaries, gamified learning, progress).
- API Interfaces: REST endpoints for communication between front-end and back-end.
- Plugin Interfaces: APIs to integrate AURELIA into external LMS platforms.

4.2.5 Component-Level Design

Component-level design involves breaking down the architecture into smaller, reusable units:

- Summarization Module: Processes uploaded content using NLP.
- Flashcard Module: Generates question-answer pairs from summaries.
- Audio/Video Module: Converts summarized text into speech and animations.
- Gamification Engine: Assigns XP, awards badges, and tracks progress.
- User Management Module: Handles authentication, history, and preferences.

4.3 The System Layout / Block Diagram

The system layout of AURELIA illustrates the interaction among its core layers: frontend, backend, AI/ML modules, and database, which collectively deliver text summarization, multimedia explanations, gamification, and user management features. As presented in Figure 4.2, the system layout illustrates the arrangement of AURELIA's key modules (Generated using DiagramGPT).

The design follows a layered modular architecture, where each layer communicates through well-defined interfaces. This modularity enhances scalability, maintainability, and interoperability while ensuring seamless LMS integration. The block diagram demonstrates the flow of data and control, beginning from user requests at the frontend, processing at the backend and AI modules, and returning structured outputs for learner interaction.

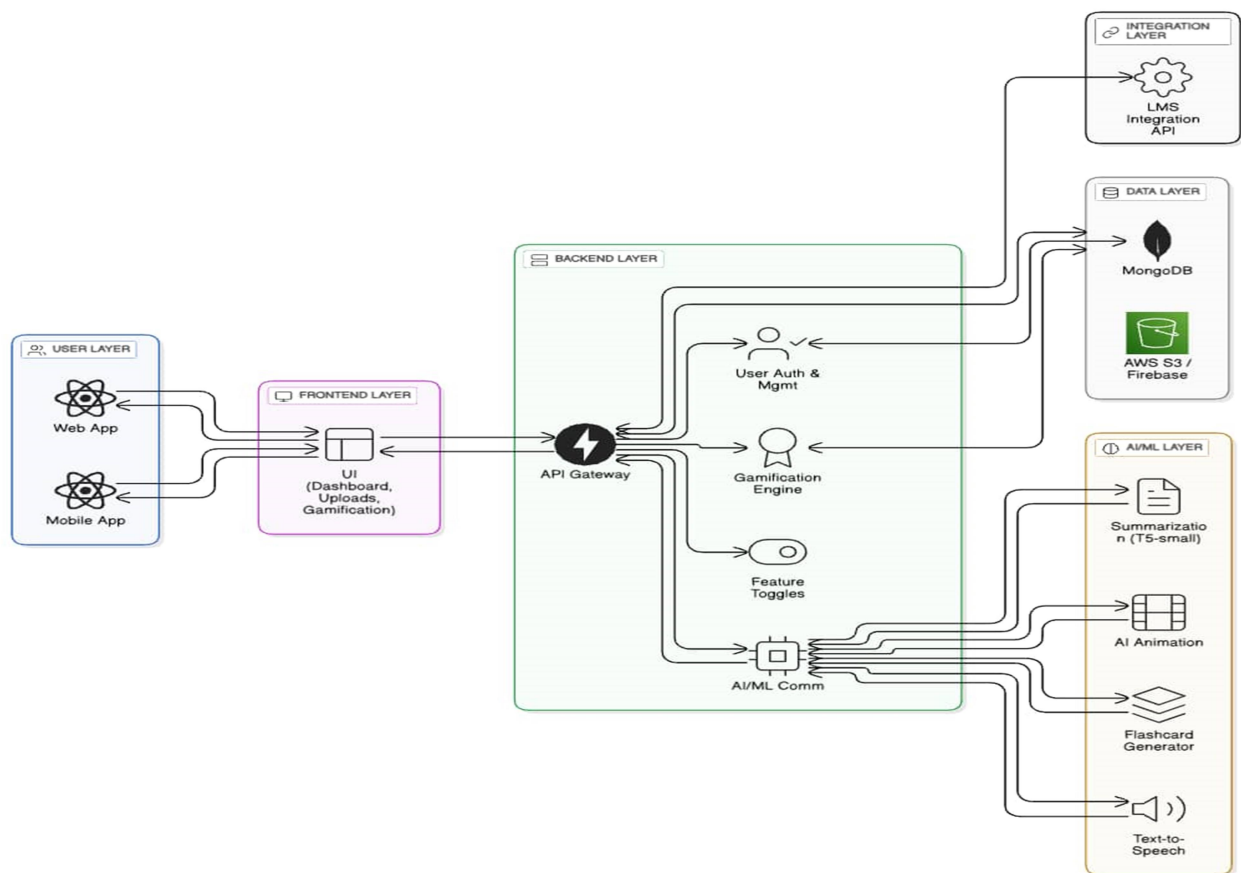


Figure 4.2: System layout of AURELIA

4.3.1 Components of the System Layout

4.3.1.1 User Layer (Learners/Instructors)

- i. Users interact via the Web App (React) or Mobile App (React Native).
- ii. Core features include document upload, summary generation, multimedia access, gamification, and activity history.

4.3.1.2 Frontend Layer (Presentation)

- i. Provides the User Interface (UI/UX) for interaction.
- ii. Sends HTTPS/REST requests to the backend server.
- iii. Displays AI outputs such as summaries, flashcards, audio narrations, videos, and gamification dashboards.

4.3.1.3 Backend Layer (Application Server, FastAPI)

- i. Manages communication between frontend and AI/ML modules.
- ii. Implements core logic such as authentication, gamification engine, and learning progress tracking.
- iii. Provides REST APIs to support standalone usage and integration into LMS platforms.

4.3.1.4 AI/ML Layer

- i. Summarization Module: Processes text using the fine-tuned T5-small model.
- ii. Flashcard Generator: Automatically produces Q&A pairs for reinforcement learning.
- iii. Audio Module: Converts summaries into speech using Text-to-Speech (TTS).
- iv. Video Module: Transforms summaries into explanatory video content.

4.3.1.5 Data Layer (Database and Storage)

- i. MongoDB stores user profiles, learning history, and gamification logs.
- ii. Cloud object storage (AWS S3 or Firebase) manages uploaded files and AI-generated media such as audio, video, and flashcards.

4.3.1.6 Integration Layer

- i. Provides APIs that allow smooth integration with Learning Management Systems (LMS).
- ii. Ensures compatibility with institutional platforms without major system redesign

4.3.3 Associated Explanation

The block diagram represents AURELIA's system layout as a pipeline of interactions:

- i. A learner uploads a file or requests a summary through the frontend interface.
- ii. The request is routed to the backend server (FastAPI), which validates the request and forwards it to the AI/ML layer.
- iii. The AI/ML layer processes the content (summarization, flashcard generation, TTS, or video synthesis).
- iv. Results are stored in the database and storage layer for future retrieval.
- v. Processed outputs (summaries, audio, video, flashcards) are returned to the frontend, where they are displayed to the learner.
- vi. The gamification engine updates progress points, awards badges, and maintains history logs.
- vii. The integration layer ensures AURELIA can act as a plug-in for LMS systems

4.4 The system flowcharts / Unified Modelling Language diagrams/Data flow diagrams

4.4.1: Data flow (Level 1)

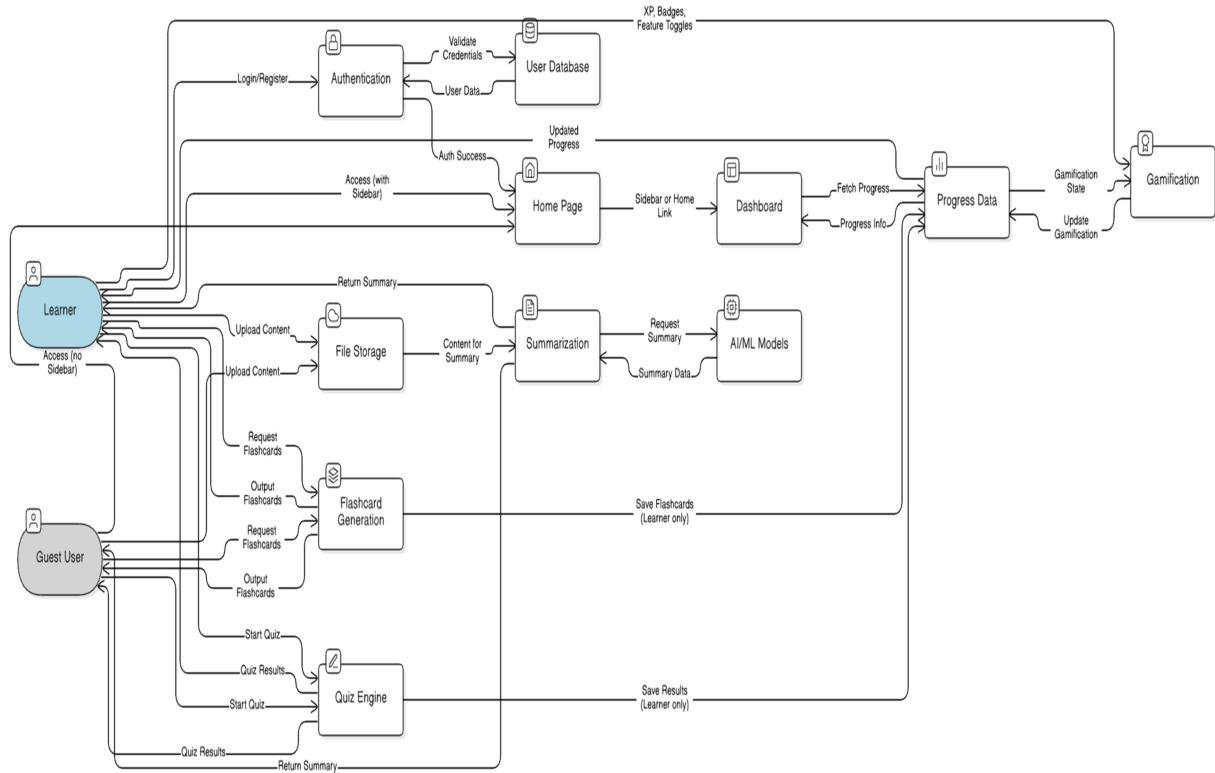


Figure 4.3: Data flow diagram (Level 1)

As illustrated in Figure 4.3 above, the Level 1 Data Flow Diagram outlines the movement of data between system components (Generated using DiagramGPT).

The Data Flow Diagram (DFD Level 1) shows how data moves between external users, internal processes, and storage systems.

External Entities: Learner (Registered User), Guest User.

Processes: Authentication, Home Page, Dashboard (learners only), Summarization, Flashcard Generation, Quiz Engine, Gamification (planned).

Data Stores: User Database (MongoDB), File Storage (Firebase/AWS S3), Progress Data (MongoDB), AI/ML Models (Hugging Face).

Flows:

Learner ↔ Authentication ↔ User Database

Learner → Home Page (with Sidebar) → Dashboard (optional)

Guest → Home Page (no Sidebar)

Both learners and guests → Upload content → File Storage → Summarization → AI/ML Module → Results returned

Flashcard and quiz results stored in Progress Data for learners only

Gamification progress stored in Progress Data (planned)

4.4.2: Use Case

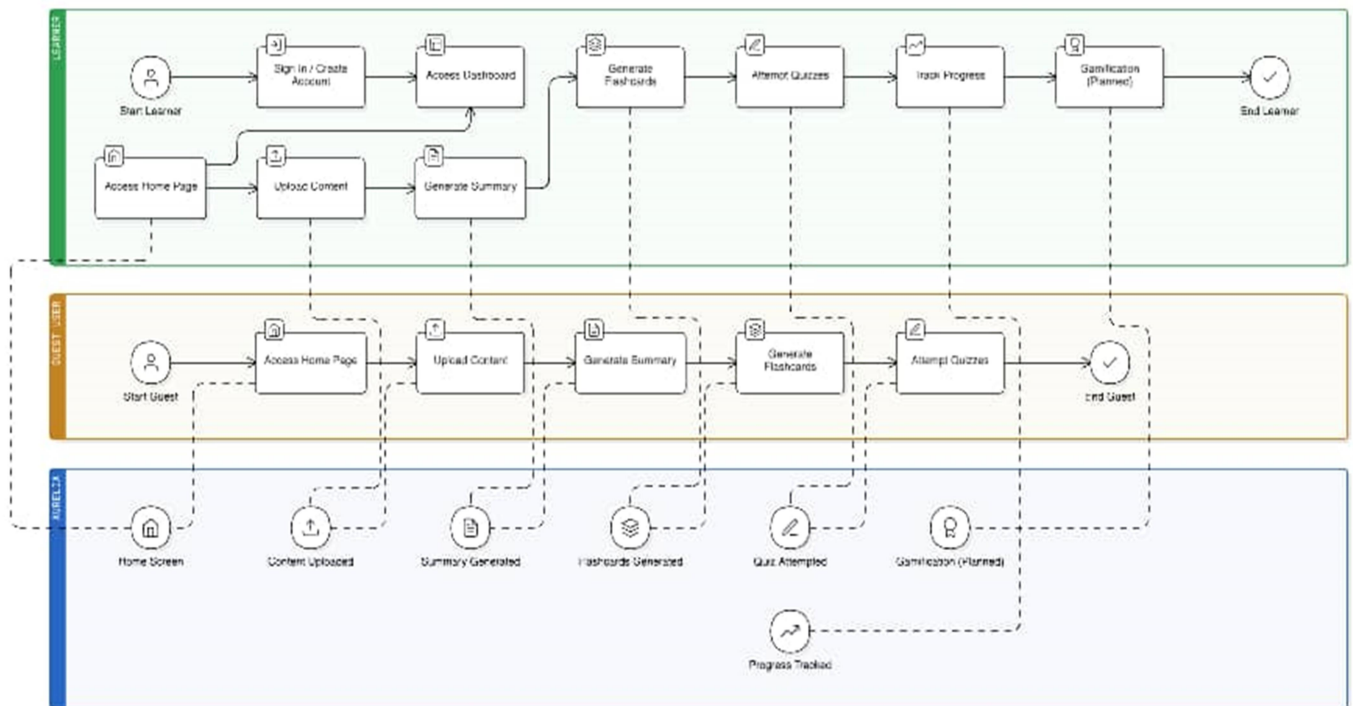


Figure 4.4: Use case diagram

The interactions between users and system functionalities are presented in Figure 4.4 above (Generated using DiagramGPT).

The Use Case diagram depicts the main functions of AURELIA and the interactions available to both registered learners and guest users. All users begin from the Home Page, with learners having additional privileges such as access to the Dashboard, progress tracking, and gamification features.

Learners: Can create an account, sign in, access the Home Page (with a sidebar), navigate to the Dashboard, and use features such as content summarization, flashcard generation, quizzes, and gamification (planned).

Guest Users: Can only access the simplified Home Page (no sidebar) and use summarization, flashcards, and quizzes without progress persistence.

4.4.3: Activity Flow

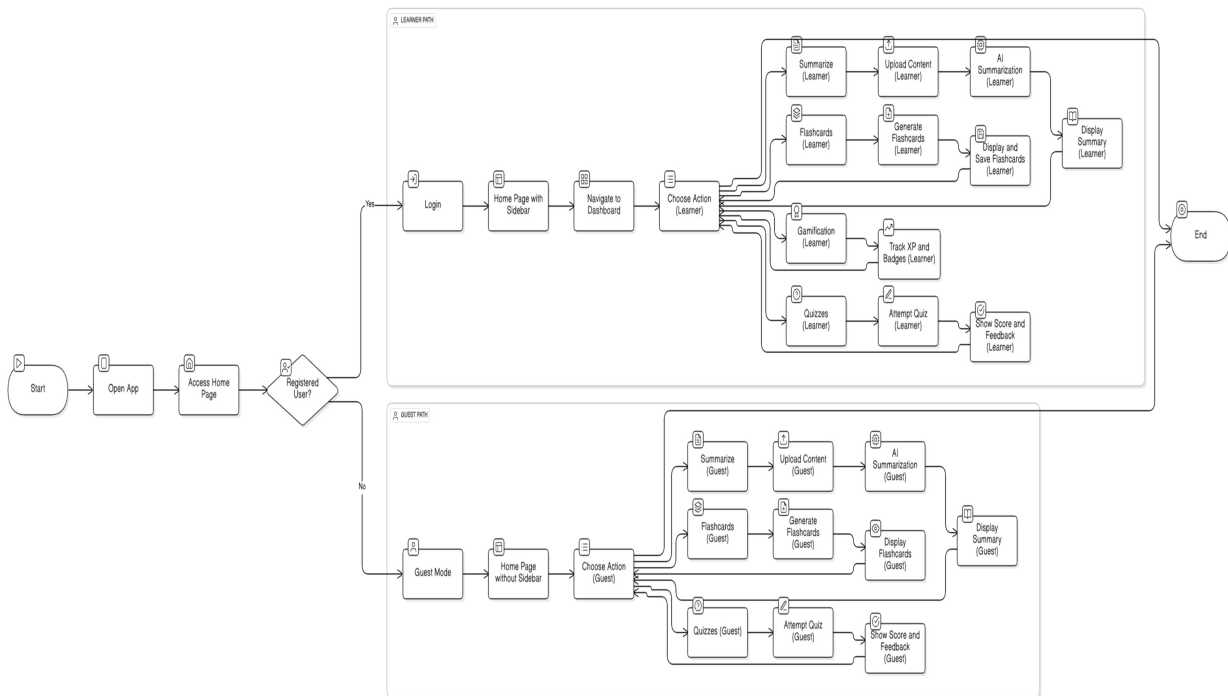


Figure 4.5: Activity diagram

As shown in Figure 4.5 above, the activity diagram details the step-by-step flow of processes within the system (Generated using DiagramGPT).

The Activity Diagram illustrates the workflow from launching the application to engaging with specific features. The Home Page acts as the first screen for both learners and guests, but it differs in functionality:

Learners see the Home Page with a sidebar that links to the Dashboard. From the Dashboard or Home, they can choose features such as summarization, flashcards, quizzes, or gamification (planned).

Guest Users see the Home Page without a sidebar. They can access summarization, flashcards, and quizzes directly from the Home interface.

For each feature:

Summarization: Users upload content → AI/ML module processes it → system returns a summary.

Flashcards: Users generate flashcards; results are saved if the user is a learner.

Quizzes: Users attempt a quiz and receive instant feedback; results are stored if the user is a learner.

Gamification: Learners accumulate XP/badges and toggle gamified modes (planned feature).

4.5 Application Module

The AURELIA system is designed as a modular application where each component handles a specific function but integrates seamlessly with others. The modular architecture enhances scalability, maintainability, and future expansion. The key modules are:

4.5.1 Authentication Module

During development, a secure signup, verification and login script was created to validate user credentials (see Appendix A for the complete code).

- i. Handles user registration, login, and session management.
- ii. Supports guest access without authentication.
- iii. Uses secure token-based authentication (JWT)

4.5.2 Home Module

Figure 4.6 shows the homepage of AURELIA, which provides users with access to the platform's key features.

- i. First point of interaction for all users.
- ii. Displays system features and provides navigation.

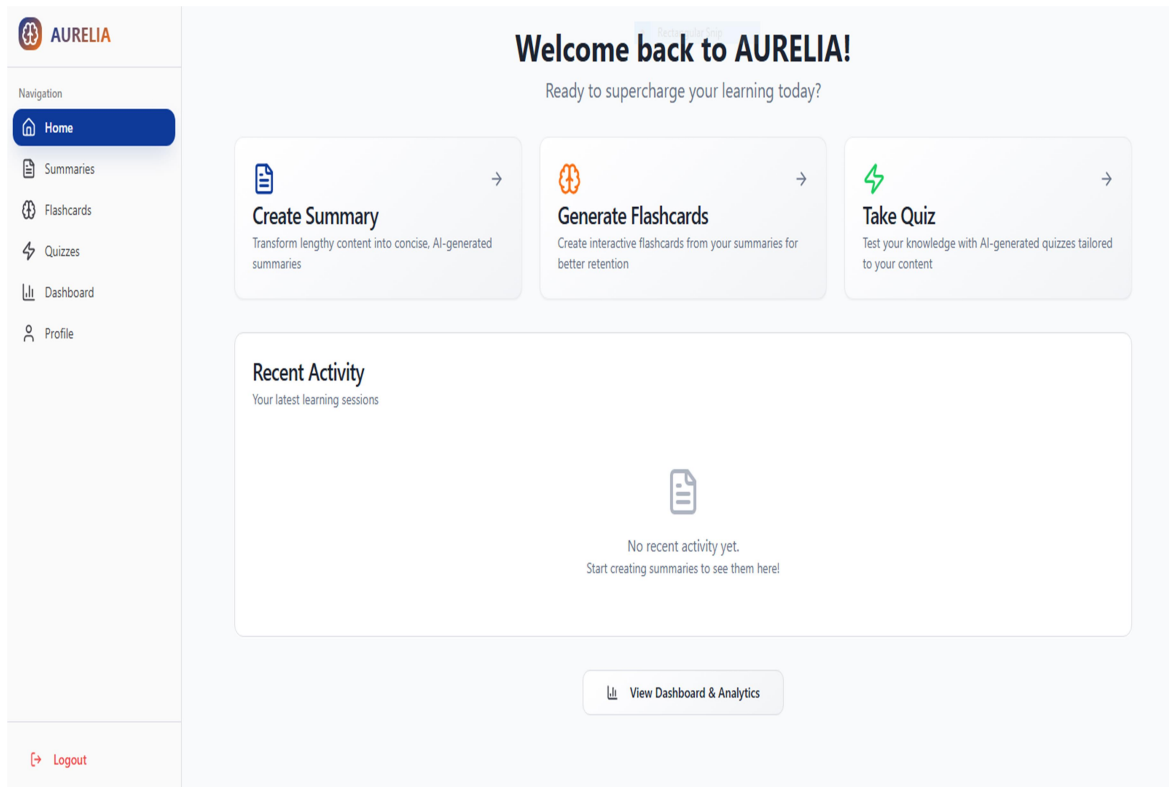


Figure 4.6: AURELIA homepage

4.5.3 Dashboard Module (Learner only)

The main interface of AURELIA is presented in Figure 4.1 above, which enables learners to navigate and access tools.

- i. Provides an overview of user progress, past summaries, quizzes, and flashcards.
- ii. Serves as a control centre for gamification and progress tracking.

4.5.4 Summarization Module

As shown in Figure 4.7 below, the summarization input page allows learners to provide content for automatic summarization.

- i. Accepts uploaded documents or text.
- ii. Sends content to AI/ML services (Hugging Face models).
- iii. Returns concise summaries to users.

The screenshot displays the 'Summaries' section of the AURELIA application. On the left is a navigation sidebar with links for Home, Summaries (active), Flashcards, Quizzes, Dashboard, and Profile. The main content area is titled 'Summaries' with the subtitle 'Manage your AI-generated summaries'. It features a '+ New Summary' button in the top right. Below this is a 'Create New Summary' form with the instruction 'Enter your content below and let AI generate an intelligent summary'. The form includes a 'Title' field with a placeholder 'Give your content a title...', a 'Content' text area with a placeholder 'Paste your content here (articles, notes, research papers, etc.)' and an 'Upload file' button, and a 'Summary length' dropdown set to 'Medium'. There are also 'Tone' and 'Simplify?' dropdowns, with the latter having a checkbox for 'Simplify language for easier understanding'. 'Generate Summary' and 'Cancel' buttons are at the bottom of the form. A tip below the form states: 'Tip: Upload a .txt/.md/.html/.json file or paste the text directly. PDF/doc parsing isn't supported in this client demo.' Below the form is a 'History' section showing two generated summaries: 'Machine Learning Fundamentals' (dated 1/15/2024) and 'React Performance Optimization' (dated 1/14/2024). A 'Logout' button is located in the bottom left of the sidebar.

Figure 4.7: Summarization input page

4.5.5 Flashcard Module

Figure 4.8 demonstrates the flashcard generation feature of AURELIA, which produces study cards for revision.

- i. Generates flashcards from summarized text.
- ii. Supports study sessions; results are saved for learners.

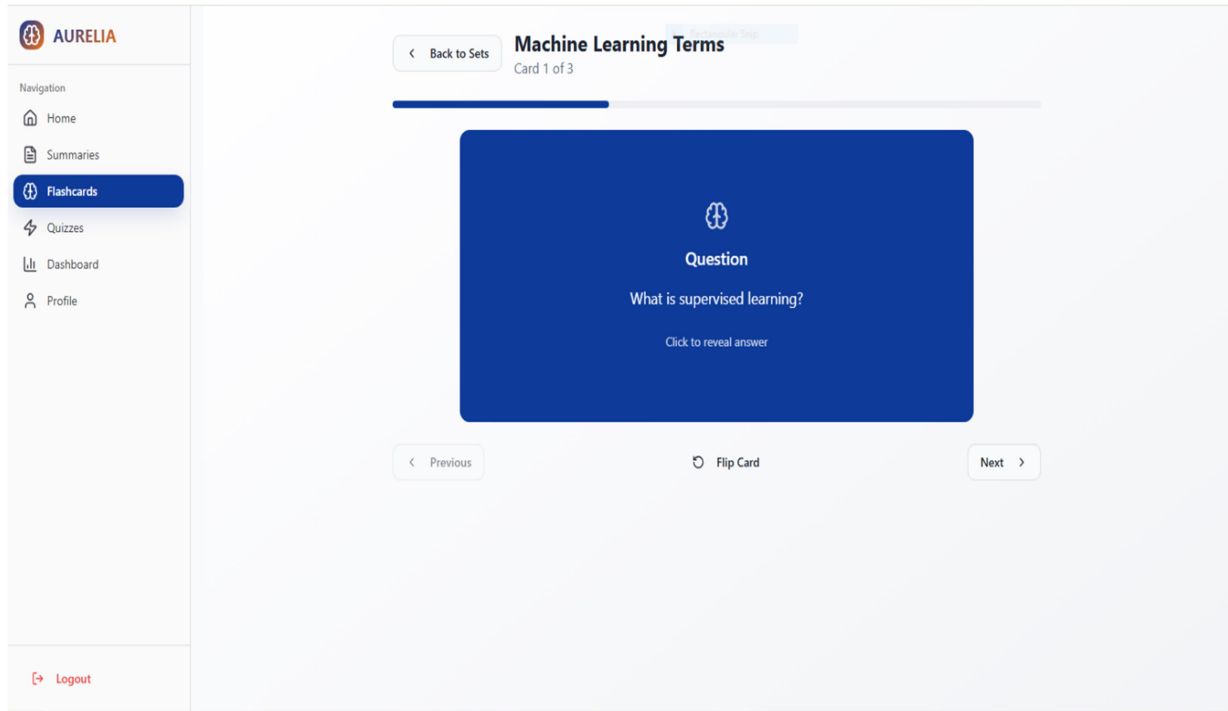


Figure 4.8: Flashcard generation view

4.5.6 Quiz Module

As illustrated in Figure 4.9, the quiz module result interface displays learner performance after assessments.

- i. Produces quizzes dynamically from input or summarized text.
- ii. Provides feedback to both guest and learner users.
- iii. Learners' results are stored in the database.

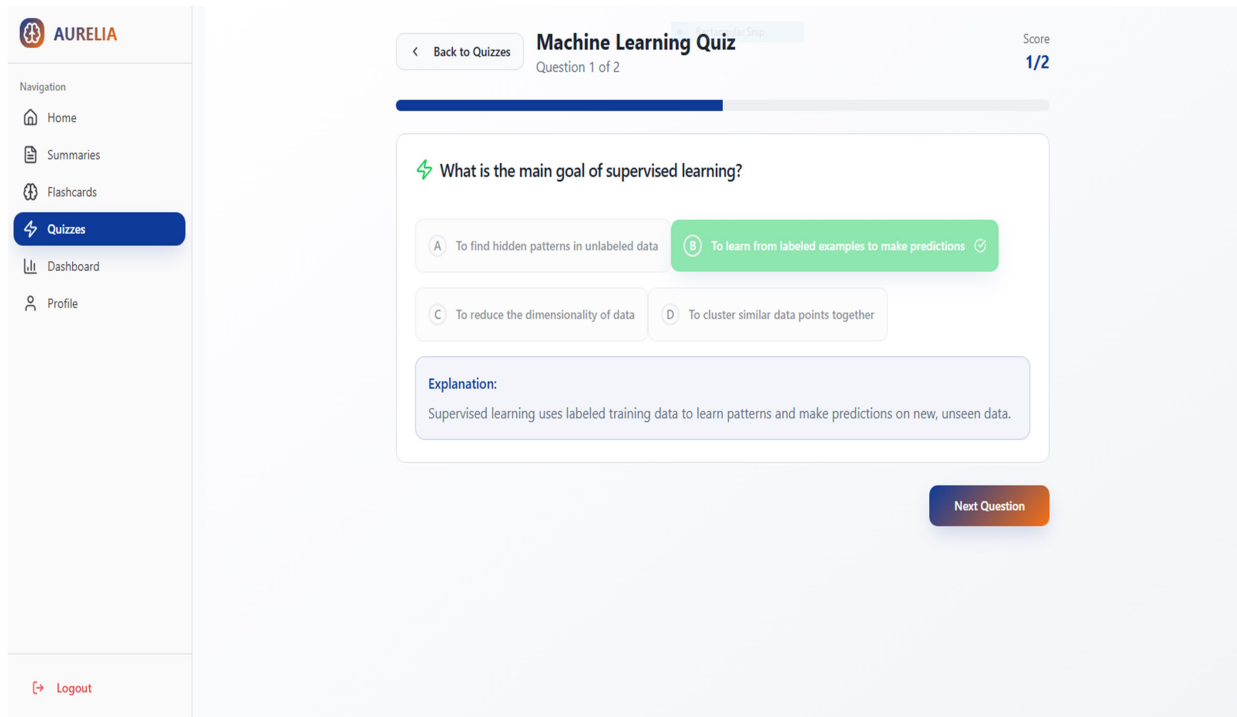


Figure 4.9: Quiz module result interface

4.5.7 Gamification Module (planned)

- i. Tracks XP, awards badges, and manages feature toggles.
- ii. Intended to enhance user engagement and motivation.

4.6 Database Design

Figure 4.10 presents the database design, showing the structure and relationships among the key entities (Generated using Mermaid Live Editor).

The system uses a NoSQL database (MongoDB) for flexibility in storing unstructured and semi-structured learning data. Firebase/AWS S3 is employed for file storage.

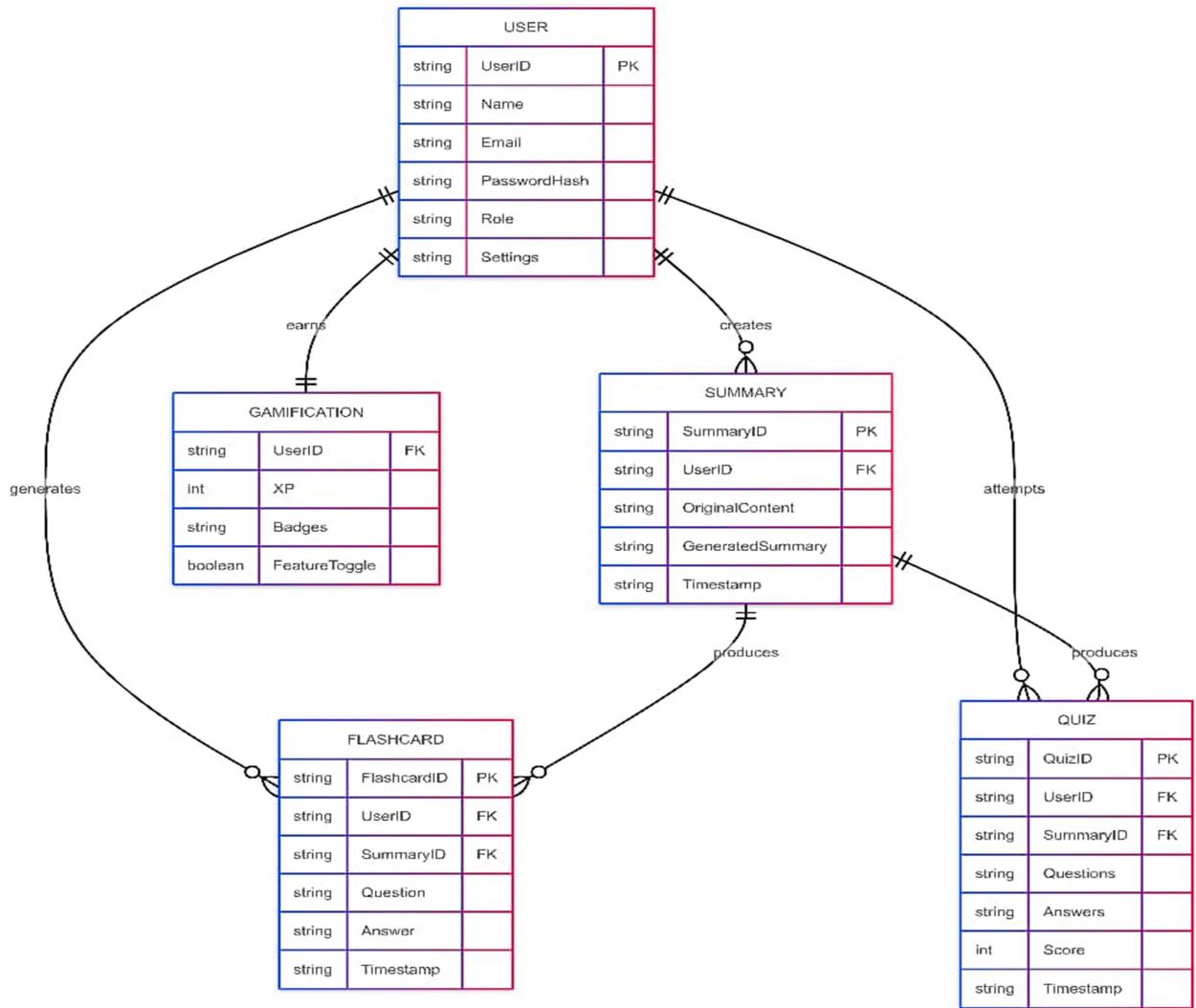


Figure 4.10: Database design

4.6.1 Entities and Collections

i. Users Collection

Attributes: UserID, Name, Email, PasswordHash, Role (Learner/Guest), Settings.

ii. Summaries Collection

Attributes: SummaryID, UserID (optional for guest = null), OriginalContent, GeneratedSummary, Timestamp.

iii. Flashcards Collection

Attributes: FlashcardID, UserID (optional), SummaryID, Question, Answer, Timestamp.

iv. Quizzes Collection

Attributes: QuizID, UserID (optional), Questions, Answers, Score, Timestamp.

v. Gamification Collection (planned)

Attributes: UserID, XP, Badges, FeatureToggle.

vi. File Storage (Firebase/AWS S3)

Stores uploaded files, linked to UserID and SummaryID.

4.6.2 Relationships

- i. A User can have many Summaries, Flashcards, and Quizzes.
- ii. Summaries may generate multiple Flashcards.
- iii. Gamification records are linked to Learner profiles only.

4.7 Hardware and Software Requirements

The hardware and software requirements of AURELIA are determined by its modular structure and the AI-driven features it provides. While the current MVP focuses on Summarization, Flashcards, Quizzes, Home, and Dashboard, additional provisions are made for the Gamification module, which is under development.

4.7.1 Hardware Requirements

Minimum (for end users):

- i. Device: Smartphone (Android 9.0+ / iOS 13.0+) or PC (Dual-core processor, 2 GB RAM).
- ii. Storage: At least 200 MB free.
- iii. Internet: Stable broadband or mobile data connection.

4.7.2 Recommended (for gamification and smooth performance):

- i. Device: Quad-core processor, 4 GB+ RAM, GPU support for 3D rendering (for Unity WebGL games).
- ii. Storage: 500 MB free.
- iii. Internet: Reliable broadband (≥ 10 Mbps).

4.7.3 Server requirements (deployment):

- i. CPU: Quad-core (or higher) cloud instance.
- ii. RAM: 8 GB+.
- iii. Storage: Scalable cloud storage (MongoDB Atlas + file storage, e.g., AWS S3 or Firebase).
- iv. GPU: Optional, for training or deploying ML models.

4.7.4 Software Requirements

- i. Frontend: React (Web), React Native (Mobile).
- ii. Backend: FastAPI (Python-based REST framework).
- iii. Database: MongoDB Atlas (cloud-based NoSQL).
- iv. AI/ML: Python with Hugging Face Transformers.
- v. Gamification (planned): Unity WebGL for interactive 3D games; optional lightweight 2D frameworks such as Pygame.
- vi. Hosting: Cloud deployment with CI/CD support.

4.8 Input and Output Requirements

The system requires different types of inputs and generates outputs depending on the selected feature. Inputs are primarily user-generated content, while outputs are AI-processed or system-generated learning aids.

4.8.1 Inputs

Summarization: Uploaded documents (PDF, DOCX, TXT) or raw text.

Flashcards: Summarized text (system-generated input).

Quizzes: Summarized text or flashcards.

Authentication: Login credentials for registered users.

Gamification (planned): Learner interactions, quiz results, session activity logs.

4.8.2 Outputs

Summarization: Concise, AI-generated summaries.

Flashcards: Question-answer flashcards linked to summaries.

Quizzes: AI-generated questions with scores and feedback.

Dashboard: Progress tracking saved results.

Gamification (planned): XP points, badges, progress levels, leaderboards.

4.9 Choice and Justification of Programming Language

The programming languages and frameworks for AURELIA were selected based on their ability to meet system requirements, ensure scalability, and integrate effectively with AI/ML services.

i. Frontend: React / React Native

React and React Native were chosen for the frontend because they provide a component-based architecture, support cross-platform development, and have a large ecosystem of community-driven libraries. Their efficiency in building responsive and interactive interfaces makes them well suited for features such as summarization, flashcards, and quizzes (Meta Open Source, 2023).

ii. Backend: FastAPI (Python)

FastAPI was selected for the backend because it offers high performance, asynchronous request handling, and automatic OpenAPI documentation. Importantly, it integrates natively with Python's machine learning ecosystem, making it ideal for handling AI-driven tasks within AURELIA (Ramírez, 2023).

iii. Database: MongoDB (NoSQL)

MongoDB was chosen as the database due to its ability to handle semi-structured and unstructured data, such as summaries, quizzes, and flashcards. Its schema flexibility supports future feature additions, including gamification. Additionally, MongoDB Atlas ensures cloud scalability and reliability (MongoDB Inc., 2023).

iv. AI/ML Integration: Python with Hugging Face Transformers

Python, combined with Hugging Face Transformers, was chosen for AI/ML integration because of its rich ecosystem of natural language processing libraries. Hugging Face provides access to state-of-the-art models, simplifying the deployment of summarization, flashcard generation, and quiz creation (Hugging Face, 2023).

v. Gamification (Planned): Unity WebGL

For the planned gamification module, Unity WebGL was selected as the primary framework for delivering interactive 3D learning games. Unity’s cross-platform support ensures that games can be seamlessly embedded within the web and mobile environments of AURELIA. Unity WebGL also allows exporting games directly to browsers, eliminating the need for additional installations and ensuring accessibility for learners.

4.10 Changeover Procedure

The changeover procedure defines how learners transition from manual study approaches to adopting AURELIA. Since the system is being introduced as a new deployment rather than replacing an existing automated system, the focus is on ensuring smooth adoption and minimizing risk.

4.10.1 Comparison of Changeover Methods

Table 5 below compares the various changeover methods that can be adopted by AURELIA

Table 5: Comparison of Changeover Methods

Method	Description	Advantages	Disadvantages
Direct	The old system is stopped, and the new system is introduced immediately.	Quick and simple implementation.	High risk of failure; no fallback if problems occur.
Parallel	Both systems are run together until the new one is stable.	Safer transition with fallback option.	Resource-intensive and potentially confusing for users.
Pilot	The new system is deployed to a small group before full rollout.	Limits risks and allows early issue detection.	May cause delays; requires careful selection of test users.

Method	Description	Advantages	Disadvantages
Phased	The new system is introduced in stages, feature by feature.	Lower risk, gradual adoption, feedback-driven improvement.	Slower deployment; requires detailed planning.

4.10.2 Adopted Changeover Method: Phased Changeover

The phased method is adopted for AURELIA. Since the application is modular, it is practical to introduce features such as summarization, flashcards, and quizzes first, followed by gamification. This gradual rollout reduces risks, allows time for testing, and ensures users adapt to each module at their own pace (Laudon & Laudon, 2022).

4.10.3 Implementation Steps

- i. **Pilot Testing:** Initial deployment of MVP features (Summarization, Flashcards, Quizzes, Dashboard) to a limited group of learners.
- ii. **Incremental Rollout:** Expanding availability to a larger audience based on pilot feedback.
- iii. **Full Deployment:** Releasing all MVP modules to the general user base.
- iv. **Future Expansion:** Integrating gamification once it is fully developed and tested.
- v. **Monitoring and Review:** Ongoing evaluation of performance, user engagement, and error reporting.

4.10.4 Benefits of the Phased Approach:

- i. Reduces risk compared to direct changeover.
- ii. Provides room for user feedback and iterative refinement.
- iii. Ensures a smoother adoption curve for both learners and guest users.

CHAPTER FIVE

SYSTEM IMPLEMENTATION

5.1 Introduction

This chapter presents the implementation of the AURELIA system. It details how the design specifications were translated into a functional prototype, the tools and frameworks employed, and the testing procedures carried out to ensure system reliability. The system was implemented in incremental stages, starting with the MVP features:

- i. Dashboard
- ii. Home Page
- iii. Summarization Module
- iv. Flashcards Module
- v. Quiz Module

5.2 System Implementation

The implementation phase was carried out using the selected tools and technologies as outlined in Chapter Four.

5.2.1 Frontend (React & React Native)

A responsive web interface was built with React, while a mobile version was developed with React Native. Both platforms support modular UI components and provide a consistent user experience. The home page was set to display upon login, with the sidebar accessible only to registered users, while guest users view a simplified home screen without the sidebar.

5.2.2 Backend (FastAPI)

The backend was developed using FastAPI, which serves REST APIs for the frontend. Endpoints were implemented for:

- i. Summarization requests (via Hugging Face models).

- ii. Flashcard generation.
- iii. Quiz generation and scoring.
- iv. User authentication and progress retrieval.

5.2.3 Database (MongoDB Atlas)

MongoDB was used to store user data, history of summaries, flashcards, and quiz results. For guest users, data is stored temporarily and discarded at session termination.

5.2.4 AI/ML Integration

Hugging F-ace models were integrated into the backend for summarization, flashcard creation, and quiz generation. Pre-trained models were used, with preprocessing pipelines for handling user input text and documents.

5.2.5 Gamification (Planned)

While not yet implemented, provisions were made for XP points, badges, and leaderboards within the database schema. Unity WebGL will be used for future development of interactive educational games.

5.3 Test Plan

The testing of the AURELIA system was conducted systematically to verify that all functional requirements were met. A black-box testing approach was adopted, focusing on input-output behaviour without regard to internal code execution.

5.3.1 Test Objectives

- i. Verify that core features (summarization, flashcards, quizzes, and dashboard) work as intended.
- ii. Confirm that guest users and registered users receive the appropriate interface (sidebar vs. non-sidebar).
- iii. Ensure system handles invalid inputs gracefully.

- iv. Validate that the backend services and database integrate seamlessly.
- v. Anticipate integration readiness for gamification (without deploying it yet).

5.3.2 Test Strategy

- i. Unit Testing: Individual modules (e.g., summarization endpoint, quiz generation) were tested in isolation.
- ii. Integration Testing: Interactions between frontend, backend, and database were validated.
- iii. System Testing: The system was tested to confirm compliance with user requirements.
- iv. Acceptance Testing: Simulated user scenarios were tested to ensure usability.

5.4 Test Data

The following test data sets were used to validate the system modules:

5.4.1 Summarization Module

- i. Input: A sample text document (e.g., an academic passage).
- ii. Expected Output: Concise summary (length controlled).

5.4.2 Flashcards Module

- i. Input: Summarized text.
- ii. Expected Output: A set of question-answer flashcards with key points extracted.

5.4.3 Quiz Module

- i. Input: Summarized content.
- ii. Expected Output: Multiple-choice and true/false questions, with scores and feedback.

5.4.4 User Authentication

Authentication functionality was tested extensively to ensure data security, with the underlying script shown in Appendix A.

- i. Input: Valid login credentials (registered user).

- ii. Expected Output: Access to dashboard and personalized features.
- iii. Input: Guest login.
- iv. Expected Output: Temporary session, home access without sidebar.

5.4.5 Error Handling

- i. Input: Empty text field for summarization.
- ii. Expected Output: Error prompt requesting valid input.

5.4.6 Gamification (Planned)

No test data included in the MVP phase, but database placeholders for XP, badges, and leaderboards were confirmed to accept mock values.

5.5 Test Results

The system was tested extensively based on the test plan and data described earlier. Results from the testing phase confirmed that the MVP modules are functional and meet the system requirements.

5.5.1 Summarization Module

- i. Result: Successfully generated summaries of different text inputs with controlled length.
- ii. Observation: Performance was fast for short inputs; for longer inputs, processing took slightly longer but remained within acceptable limits.
- iii. Status: Passed.

5.5.2 Flashcards Module

- i. Result: Automatically generated question–answer pairs from summarized text.
- ii. Observation: Flashcards were relevant and retained context. Some outputs required minor rephrasing for clarity.
- iii. Status: Passed.

5.5.3 Quiz Module

- i. Result: Generated multiple-choice and true/false questions with automated scoring.

- ii. Observation: Quizzes were functional, with accurate feedback on user performance.
- iii. Status: Passed.

5.5.4 User Authentication and Role Management

- i. Result: Registered users could log in and access the full dashboard with a sidebar.
Guest users could only access the home page without a sidebar.
- ii. Observation: The distinction between guest and registered modes worked as intended.
- iii. Status: Passed.

5.5.5 Error Handling

- i. Result: Invalid inputs (e.g., empty text for summarization) produced user-friendly error prompts.
- ii. Status: Passed.

5.5.6 Gamification (Planned)

- i. Result: Mock entries for XP, badges, and leaderboards were successfully stored in the database schema.
- ii. Observation: Full testing will occur once the gamification module is implemented using Unity WebGL.
- iii. Status: Deferred.

5.6 System Integration

The integration process ensured seamless communication between the system components frontend, backend, database, and AI/ML services.

- i. Frontend–Backend Integration

The React (web) and React Native (mobile) interfaces communicated with the FastAPI backend using REST APIs.

Endpoints for summarization, flashcards, quizzes, and authentication were integrated successfully.

ii. Backend–AI/ML Layer Integration

The backend called Hugging Face model APIs for summarization, flashcard, and quiz generation.

Data preprocessing (tokenization, cleaning) was done in FastAPI before sending requests.

iii. Backend–Database Integration

MongoDB Atlas was integrated to store user profiles, summaries, flashcards, quiz scores, and session history.

Temporary storage for guest users worked correctly, with automatic deletion after the session ended.

iv. System Workflow Integration

Registered users: Logged in → Home Page (with sidebar) → Dashboard → Access summarization, flashcards, quizzes.

Guest users: Opened app → Home Page (no sidebar) → Access summarization, flashcards, quizzes → Temporary session.

v. Gamification Integration (Planned)

Placeholders for gamification (XP, badges, leaderboards) were built into the database schema.

Unity WebGL was identified as the development environment for 3D gamified features.

Integration testing for this module will be performed during the next phase.

5.7 System Performance Evaluation

The performance of AURELIA was evaluated to determine how well it meets the functional and non-functional requirements defined during system design.

i Response Time

Summarization, flashcard, and quiz generation responses averaged 2–5 seconds depending on input size.

Performance was acceptable for academic and practical use, with longer texts slightly increasing processing time.

ii. Scalability

The system backend was implemented with FastAPI, which is asynchronous and lightweight, making it suitable for handling multiple concurrent requests.

Future scalability can be achieved by deploying on containerized services (e.g., Docker, Kubernetes).

iii. Reliability

Testing confirmed that the system consistently produces valid outputs for different input formats.

Temporary guest mode storage and permanent learner history both worked without data loss.

iv. Resource Utilization

The frontend (React/React Native) ran smoothly on standard devices without requiring high-end hardware.

Backend and database resource utilization remained moderate during test loads.

v. Gamification (Planned)

Preliminary database schema tests confirmed that performance could support XP and badge tracking.

Unity WebGL integration will require further optimization testing to ensure smooth performance in browser and mobile environments.

5.8 System Security

System security was considered critical during the implementation of AURELIA, particularly since it handles user accounts, progress data, and AI-generated educational content. The following mechanisms were adopted to ensure safe operation:

i. Authentication and Access Control

Registered Users: Authentication is managed using JSON Web Tokens (JWTs). Upon login, a JWT is issued, which securely encodes the user's identity and session information. Tokens are time-bound and verified with each request to ensure session validity.

Guest Users: Guest sessions are assigned temporary tokens that expire automatically at the end of a session. These tokens allow limited access (home, summarization, flashcards, and quizzes) without persistent storage.

ii. Secure Data Transmission

All communication between frontend and backend occurs over HTTPS, ensuring that sensitive data (credentials, summaries, quiz results) is encrypted during transmission and protected from interception.

iii. Input Validation and Error Handling

Input fields (text for summarization, flashcard prompts, quiz answers) are validated before processing to prevent malicious payloads, such as SQL injection or cross-site scripting (XSS).

Error messages are structured to inform the user without exposing internal system details.

iv. . Session Management

JWTs for registered users are short-lived and refreshed periodically to reduce the risk of token theft.

Guest session tokens are lightweight, with no persistence beyond a single session, ensuring data is automatically discarded when the session ends.

v. Data Protection and Database Security

MongoDB Atlas provides encryption-at-rest and role-based access control.

Access to the database is restricted via IP whitelisting and strong authentication credentials.

vi. Gamification (Planned Security Considerations)

When the gamification module is introduced, server-side validation will be enforced for XP and badge assignments to prevent tampering.

Anti-cheating mechanisms (e.g., event logging, anomaly detection) will be incorporated into game-based learning interactions.

5.9 System Documentation

Proper documentation was created to support system development, deployment, and user adoption.

5.9.1 Technical Documentation

- i System Architecture Documentation: Outlines frontend, backend, database, and AI/ML service integration.
- ii API Documentation: Provides details on available endpoints (summarization, flashcards, quizzes, authentication).
- iii Database Documentation: ER diagrams and schema descriptions.
- iv Gamification Provisions: Placeholder documentation describing XP/badge tracking schema and future Unity WebGL integration.

5.9.2 User Documentation

- i User Guide: Explains how to sign in, use guest mode, and access modules.
- ii Feature Walkthroughs: Step-by-step guides for uploading text, generating summaries, flashcards, and quizzes.
- iii Error Handling Guide: Explains common system messages and how to resolve them.

5.9.3 Maintenance Documentation

Includes deployment instructions, dependencies, and troubleshooting procedures for backend and frontend services.

5.10 System Maintenance

System maintenance is essential to ensure continued reliability, adaptability, and scalability of AURELIA.

- i Corrective Maintenance

Bug fixes and error handling will be performed whenever faults are detected during user interactions.

- ii Adaptive Maintenance

The system will be updated to remain compatible with changes in third-party tools (e.g., Hugging Face APIs, MongoDB Atlas). Unity WebGL gamification will be added as an adaptive upgrade in future phases.

- iii Perfective Maintenance

Enhancements will include improvements in summarization accuracy, better flashcard phrasing, and quiz diversity. The gamification module will be optimized for smoother gameplay and user engagement.

- iv Preventive Maintenance

Regular monitoring of server performance, database health checks, and system logs. Security audits to detect and mitigate potential vulnerabilities before exploitation.

CHAPTER SIX

SUMMARY, CONCLUSION AND RECOMMENDATION

6.1 Summary of Achievements

This project set out to design and implement AURELIA (AI-powered User-Centric Retention-boosting Engaging Learning Immersive Assistant), a platform that leverages artificial intelligence to enhance the learning experience through summarization, flashcards, and quizzes.

The following achievements were recorded:

6.1.1 System Design and Implementation

- i Developed a modular system architecture comprising frontend (React, React Native), backend (FastAPI), AI/ML integration (Hugging Face models), and database (MongoDB Atlas).
- ii Designed a user-friendly interface with distinct experiences for registered users (with dashboard and sidebar navigation) and guest users (simplified home view without sidebar).

6.1.2 Core Functionalities (MVP)

- i Summarization Module: Successfully implemented text summarization using pre-trained AI models.
- ii Flashcards Module: Automatically generated question-answer flashcards from summaries for self-study.
- iii Quiz Module: Provided dynamic quizzes with scoring and feedback to reinforce learning.
- iv Dashboard and Home Interface: Enabled users to track activities (learners) and access key features efficiently.

6.1.3 System Integration and Testing

- i Integrated frontend, backend, and database layers seamlessly with REST APIs.
- ii Validated system functionality through structured test plans, test data, and test results.
- iii Confirmed performance reliability, error handling, and scalability potential.

6.1.4 Security and Maintenance

- i Implemented secure authentication with JWTs for registered users and temporary tokens for guests.
- ii Enforced HTTPS encryption and input validation to safeguard data and prevent malicious activity.
- iii Outlined a comprehensive system maintenance plan, covering corrective, adaptive, perfective, and preventive measures.

6.1.5 Provisions for Future Expansion

- i Designed database placeholders and architectural flexibility for gamification features (XP, badges, leaderboards).
- ii Selected Unity WebGL for future interactive game development to further engage learners.

6.1.6 Overall Achievement:

The project successfully delivered a working MVP that demonstrates how AI can support personalized, engaging, and retention-boosting learning.

6.2 Conclusion

The development of AURELIA demonstrates that artificial intelligence can be effectively harnessed to improve the learning experience through automation and personalization. By providing tools such as summarization, flashcards, and quizzes, learners are empowered to process information more efficiently and engage in active recall, thereby boosting retention.

The system's design ensures accessibility for both guest users (casual learners) and registered users (with full progress tracking), striking a balance between flexibility and personalization. Security considerations, including JWT-based authentication, temporary session tokens, HTTPS, and input validation, ensure that the system remains safe and reliable.

Although the gamification module is still under development, its integration into the architecture underscores the project's long-term vision of making learning not only effective but also immersive and enjoyable. With Unity WebGL planned for deployment, the system will evolve into a more dynamic platform capable of blending AI-driven personalization with interactive game-based learning.

In conclusion, AURELIA's MVP has demonstrated the feasibility of combining AI-powered content generation with interactive learning features. It lays a strong foundation for future enhancements, particularly the integration of gamification, which is expected to further elevate learner engagement and motivation.

6.3 Major Contribution to Knowledge

This research contributes to knowledge by:

- i. Developing and evaluating a fine-tuned summarization model (T5-small) tailored for educational text, supporting tone, readability, and length customization.
- i. Demonstrating how Unity WebGL gamification can be tightly integrated with NLP models for enhanced learner engagement.
- ii. Introducing a hybrid platform that functions both as a standalone web/mobile app and as an LMS plug-in, filling a gap left by existing systems.
- iii. Providing a framework for combining multimodal outputs (text, audio, video) with progress tracking for inclusive learning.
- iv. Offering practical insights into backend-first AI system design using FastAPI and Hugging Face pipelines, showing a low-cost approach for deploying educational AI.

6.4 Recommendations

Based on findings, the study recommends:

- i Institutions should adopt AI-powered summarization tools to reduce information overload and improve comprehension.

- ii Educators should integrate gamification selectively (toggle-based) to suit different learner preferences.
- iii Future versions of AURELIA should incorporate adaptive personalization using reinforcement learning for deeper learner modelling.
- iv Data privacy and ethical safeguards should be prioritized when deploying AI in education.
- v Further research should explore extending AURELIA with VR/AR for immersive educational experiences.
- vi AURELIA leverages authentication as a primary access control measure (Appendix A), which can be strengthened with multi-factor verification in future upgrades.

REFERENCES

- Aljohani, N. R. (2023). The role of AI-powered summarization tools in reducing information overload in education. *Education and Information Technologies*, 28(7), 8811–8830. <https://doi.org/10.1007/s10639-023-11882-6>
- Aloizou, A. (2025). Multimodal inclusivity in digital education: Bridging accessibility gaps. *Journal of Inclusive Learning*, 12(1), 45–59.
- Chen, J., Wang, L., & Zhang, M. (2020). AI and gamification: Adaptive learning strategies for engagement. *Computers & Education*, 157, 103982. <https://doi.org/10.1016/j.compedu.2020.103982>
- Deterding, S., & Björk, S. (2020). Gamification and education: A review of empirical studies. *Simulation & Gaming*, 51(1), 9–35. <https://doi.org/10.1177/1046878119884635>
- Dicheva, D., Dichev, C., Agre, G., & Angelova, G. (2015). Gamification in education: A systematic mapping study. *Educational Technology & Society*, 18(3), 75–88.
- Guo, J., Li, X., & Xu, H. (2021). Adaptive summarization for personalized learning. *Journal of Educational Data Mining*, 13(2), 55–71.
- Hamari, J., Koivisto, J., & Sarsa, H. (2014). Does gamification work? A literature review of empirical studies on gamification. *Proceedings of the 47th Hawaii International Conference on System Sciences*, 3025–3034. IEEE. <https://doi.org/10.1109/HICSS.2014.377>
- Heliyon. (2023). Gamification outcomes in higher education: A meta-analysis. *Heliyon*, 9(4), e14567. <https://doi.org/10.1016/j.heliyon.2023.e14567>
- Hugging Face. (2024). Hugging Face Transformers Documentation. Retrieved from <https://huggingface.co/docs>
- Lang, P., Meyer, T., & Schmidt, R. (2023). Real-time analytics in gamified learning systems. *Journal of Learning Analytics*, 10(2), 120–139.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *Proceedings of ACL 2020*, 7871–7880. <https://doi.org/10.18653/v1/2020.acl-main.703>
- Liu, Y., & Lapata, M. (2019). Text summarization with pretrained encoders. *Proceedings of EMNLP-IJCNLP 2019*, 3730–3740. <https://doi.org/10.18653/v1/D19-1387>

- Liu, Y., Zheng, Y., & Chen, Q. (2023). Transformer-based domain-specific summarization for academic learning. *Computers & Education*, 190, 104617. <https://doi.org/10.1016/j.compedu.2023.104617>
- Lomellini, F., Bianchi, M., & Rizzi, A. (2025). Empirical evaluation of multimodal inclusivity in e-learning platforms. *International Journal of Educational Technology*, 16(2), 98–115.
- Luo, L., Chen, X., & Wang, Y. (2021). Adaptive personalization in AI-driven learning platforms: A reinforcement learning approach. *International Journal of Artificial Intelligence in Education*, 31(5), 875–895. <https://doi.org/10.1007/s40593-021-00265-4>
- Martinez-Maldonado, R., Buckingham Shum, S., & Rodríguez-Triana, M. J. (2021). Extending learning management systems with AI and gamification: A design framework. *British Journal of Educational Technology*, 52(4), 1371–1387. <https://doi.org/10.1111/bjet.13086>
- Mayer, R. E. (2005). *The Cambridge handbook of multimedia learning*. Cambridge University Press.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
- Rani, S., Sharma, P., & Gill, H. (2023). Comparative analysis of transformer models for educational text summarization. *Journal of Applied Educational Research*, 7(3), 215–229.
- Rush, A. M., Chopra, S., & Weston, J. (2015). A neural attention model for abstractive sentence summarization. *Proceedings of EMNLP 2015*, 379–389. <https://doi.org/10.18653/v1/D15-1044>
- Sweller, J., Ayres, P., & Kalyuga, S. (2019). *Cognitive load theory* (2nd ed.). Springer.
- Tsai, C., Hsu, Y., & Chen, C. (2022). Gamification-enhanced summarization learning: An experimental study. *Interactive Learning Environments*, 30(5), 754–770. <https://doi.org/10.1080/10494820.2020.1799031>
- Tzimas, D., & Demetriadis, S. (2022). Ethical safeguards in AI-powered education: A framework for responsible use. *Computers & Education*, 186, 104533. <https://doi.org/10.1016/j.compedu.2022.104533>
- Webber, J., & Kim, H. (2024). Immersive VR/AR in education: Opportunities and challenges for learning engagement. *Journal of Computer Assisted Learning*, 40(2), 389–403. <https://doi.org/10.1111/jcal.12798>
- Yan, Z., Zhao, J., & Liu, W. (2023). Ethical implications of AI in educational contexts. *AI & Society*, 38(3), 1123–1138. <https://doi.org/10.1007/s00146-022-01387-y>
- Zhang, H., Li, S., & Zhou, Y. (2022). Evaluating GPT-3 and T5 for educational summarization tasks. *Computers in Human Behaviour*, 134, 107382. <https://doi.org/10.1016/j.chb.2022.107382>

Zou, D., Huang, Y., & Xie, H. (2021). A review of research on gamification in education: Emerging trends and future directions. *Educational Technology Research and Development*, 69(6), 445–464. <https://doi.org/10.1007/s11423-021-10048-6>

APPENDIX

Appendix A: User Authentication fastAPI Code Implementation

(Source: Ikede Divine Oghenekeno, 2025)

```
from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import RedirectResponse
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from passlib.context import CryptContext
from datetime import datetime, timedelta
from pymongo.errors import PyMongoError
from bson import ObjectId
from schemas.user import UserCreate, UserLogin, Token, UserOut, VerifyRequest
from models.db import users_collection, verification_tokens_collection
from services.email_service import send_verification_email
from config import Config
import logging

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

router = APIRouter(prefix="/auth", tags=["auth"])

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login", auto_error=False)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=Config.JWT_EXPIRY_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, Config.JWT_SECRET,
algorithm=Config.JWT_ALGORITHM)

async def get_current_user(token: str = Depends(oauth2_scheme)) -> UserOut:
    try:
        if not token:
            raise HTTPException(401, "No token provided")
```

```

        payload = jwt.decode(token, Config.JWT_SECRET,
algorithm=[Config.JWT_ALGORITHM])
        email: str = payload.get("sub")
        if email is None:
            raise HTTPException(401, "Invalid authentication")
        user = await users_collection.find_one({"email": email})
        if not user:
            raise HTTPException(401, "User not found")
        return UserOut(
            id=str(user["_id"]),
            username=user["username"],
            email=user["email"],
            is_verified=user.get("is_verified", False),
            join_date=user.get("join_date").isoformat() if
user.get("join_date") else None
        )
    except JWTError:
        raise HTTPException(401, "Invalid token")

@router.get("/me", response_model=UserOut)
async def get_user(current_user: UserOut = Depends(get_current_user)):
    return current_user

@router.post("/signup", response_model=Token)
async def signup(user: UserCreate):
    try:
        if await users_collection.find_one({"email": user.email}):
            raise HTTPException(400, "Email already registered")
        hashed = get_password_hash(user.password)
        user_dict = user.dict()
        user_dict["password"] = hashed
        user_dict["join_date"] = datetime.utcnow()
        user_dict["avatar"] = "😊"
        user_dict["is_verified"] = False
        result = await users_collection.insert_one(user_dict)
        logger.info(f"User created with ID: {result.inserted_id}")
        await send_verification_email(str(result.inserted_id), user.email,
user.username)
        token = create_access_token({"sub": user.email})
        return {"access_token": token}
    except PyMongoError as e:
        logger.error(f"Database error during signup: {str(e)}")
        raise HTTPException(500, f"Database error: {str(e)}")
    except Exception as e:
        logger.error(f"Error during signup: {str(e)}")
        raise HTTPException(500, f"Failed to create user: {str(e)}")

```

```

@router.post("/login", response_model=Token)
async def login(user: UserLogin):
    try:
        db_user = await users_collection.find_one({"email": user.email})
        if not db_user or not verify_password(user.password,
db_user["password"]):
            raise HTTPException(401, "Incorrect email or password")
        if not db_user.get("is_verified", False):
            raise HTTPException(403, "Email not verified. Please check your
inbox or request a new verification email.")
        token = create_access_token({"sub": db_user["email"]})
        return {"access_token": token}
    except PyMongoError as e:
        logger.error(f"Database error during login: {str(e)}")
        raise HTTPException(500, f"Database error: {str(e)}")
    except Exception as e:
        logger.error(f"Error during login: {str(e)}")
        raise HTTPException(500, f"Failed to login: {str(e)}")

@router.post("/verify")
async def verify_email(verify: VerifyRequest):
    token_doc = await verification_tokens_collection.find_one({"token":
verify.token})
    if not token_doc or token_doc["expires_at"] < datetime.utcnow():
        raise HTTPException(400, "Invalid or expired token")

    await users_collection.update_one(
        {"_id": token_doc["user_id"]},
        {"$set": {"is_verified": True}}
    )
    await verification_tokens_collection.delete_one({"token": verify.token})
    return {"message": "Email verified successfully"}

@router.get("/verify/{token}")
async def verify_email_get(token: str):
    try:
        token_doc = await verification_tokens_collection.find_one({"token":
token})
        if not token_doc or token_doc["expires_at"] < datetime.utcnow():
            logger.warning(f"Invalid or expired token: {token}")
            return
    except:
        return
    RedirectResponse(f"{Config.FRONTEND_URL}/login?error=Invalid or expired
token")

    # Verify user exists before updating
    user_id = token_doc["user_id"]
    try:

```

```

        user_id_obj = ObjectId(user_id)
    except Exception as e:
        logger.error(f"Invalid user_id format: {user_id}, error: {str(e)}")
        return
    RedirectResponse(f"{Config.FRONTEND_URL}/login?error=Invalid user ID format")

    user = await users_collection.find_one({"_id": user_id_obj})
    if not user:
        logger.error(f"User not found for ID: {user_id}")
        return RedirectResponse(f"{Config.FRONTEND_URL}/login?error=User not found")

    if user.get("is_verified", False):
        logger.info(f"User already verified: {user_id}")
        return
    RedirectResponse(f"{Config.FRONTEND_URL}/login?message=Email already verified")

    # Update is_verified
    result = await users_collection.update_one(
        {"_id": user_id_obj},
        {"$set": {"is_verified": True}}
    )

    if result.modified_count == 0:
        logger.error(f"Failed to update user {user_id}: No documents modified")
        return RedirectResponse(f"{Config.FRONTEND_URL}/login?error=Failed to verify email")

    logger.info(f"User {user_id} verified successfully")
    await verification_tokens_collection.delete_one({"token": token})
    return RedirectResponse(f"{Config.FRONTEND_URL}/login?message=Email verified successfully")
    except PyMongoError as e:
        logger.error(f"Database error during verification: {str(e)}")
        return RedirectResponse(f"{Config.FRONTEND_URL}/login?error=Database error")
    except Exception as e:
        logger.error(f"Error during verification: {str(e)}")
        return
    RedirectResponse(f"{Config.FRONTEND_URL}/login?error=Verification failed")

@router.post("/resend-verification")
async def resend_verification(email: str):
    try:

```

```

        user = await users_collection.find_one({"email": email})
        if not user:
            logger.warning(f"Resend verification attempted for non-existent
email: {email}")
            raise HTTPException(404, "User not found")
            if user.get("is_verified", False):
                logger.info(f"Resend verification attempted for already verified
user: {email}")
                raise HTTPException(400, "User already verified")
            await send_verification_email(str(user["_id"]), user["email"],
user["username"])
            logger.info(f"Verification email resent for user: {email}")
            return {"message": "Verification email sent"}
        except PyMongoError as e:
            logger.error(f"Database error during resend verification: {str(e)}")
            raise HTTPException(500, f"Database error: {str(e)}")
        except Exception as e:
            logger.error(f"Error during resend verification: {str(e)}")
            raise HTTPException(500, f"Failed to resend verification email:
{str(e)}")

```